3.1 Overview

Introduction

The process model introduced in the previous chapter assumed that a process was an executing program with a single thread of control. Virtually all modern operating systems, however, provide features enabling a process to contain multiple threads of control. Identifying opportunities for parallelism through the use of threads is becoming increasingly important for modern multicore systems that provide multiple CPUs.

In this chapter, we introduce many concepts, as well as challenges, associated with multithreaded computer systems, including a discussion of the APIs for the Pthreads, Windows, and Java thread libraries. Additionally, we explore several new features that abstract the concept of creating threads, allowing developers to focus on identifying opportunities for parallelism and letting language features and API frameworks manage the details of thread creation and management. We look at a number of issues related to multithreaded programming and its effect on the design of operating systems. Finally, we explore how the Windows and Linux operating systems support threads at the kernel level.

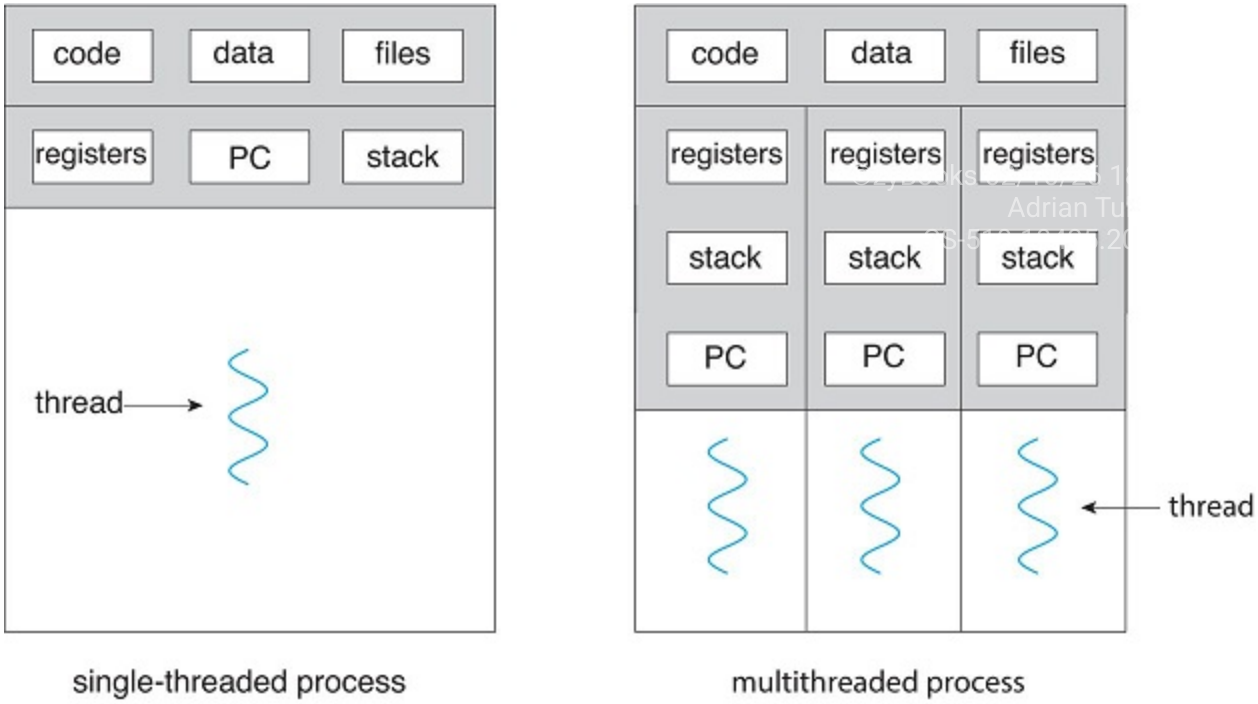
Chapter objectives

- Identify the basic components of a thread, and contrast threads and processes.
- Describe the major benefits and significant challenges of designing multithreaded processes.
- Illustrate different approaches to implicit threading, including thread pools, fork-join, and Grand Central Dispatch.
- Describe how the Windows and Linux operating systems represent threads.
- Design multithreaded applications using the Pthreads, Java, and Windows threading APIs.

Overview

A thread is a basic unit of CPU utilization; it comprises a thread ID, a program counter (PC), a register set, and a stack. It shares with other threads belonging to the same process its code section, data section, and other operating-system resources, such as open files and signals. A traditional process has a single thread of control. If a process has multiple threads of control, it can perform more than one task at a time. Figure 3.1.1 illustrates the difference between a traditional ***single-threaded*** process and a ***multithreaded*** process.

Figure 3.1.1: Single-threaded and multithreaded processes.



Motivation

Most software applications that run on modern computers and mobile devices are multithreaded. An application typically is implemented as a separate process with several threads of control. Below we highlight a few examples of multithreaded applications:

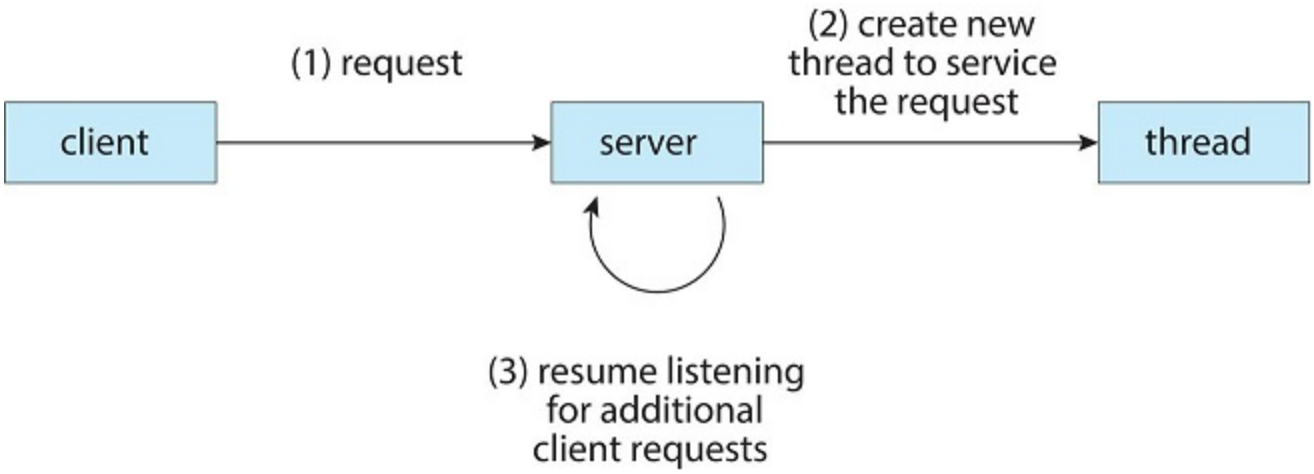
- An application that creates photo thumbnails from a collection of images may use a separate thread to generate a thumbnail from each separate image.
- A web browser might have one thread display images or text while another thread retrieves data from the network.
- A word processor may have a thread for displaying graphics, another thread for responding to keystrokes from the user, and a third thread for performing spelling and grammar checking in the background.

Applications can also be designed to leverage processing capabilities on multicore systems. Such applications can perform several CPU-intensive tasks in parallel across the multiple computing cores.

In certain situations, a single application may be required to perform several similar tasks. For example, a web server accepts client requests for web pages, images, sound, and so forth. A busy web server may have several (perhaps thousands of) clients concurrently accessing it. If the web server ran as a traditional single-threaded process, it would be able to service only one client at a time, and a client might have to wait a very long time for its request to be serviced.

One solution is to have the server run as a single process that accepts requests. When the server receives a request, it creates a separate process to service that request. In fact, this process-creation method was in common use before threads became popular. Process creation is time consuming and resource intensive, however. If the new process will perform the same tasks as the existing process, why incur all that overhead? It is generally more efficient to use one process that contains multiple threads. If the web-server process is multithreaded, the server will create a separate thread that listens for client requests. When a request is made, rather than creating another process, the server creates a new thread to service the request and resumes listening for additional requests. This is illustrated in Figure 3.1.2.

Figure 3.1.2: Multithreaded server architecture.



Most operating system kernels are also typically multithreaded. As an example, during system boot time on Linux systems, several kernel threads are created. Each thread performs a specific task, such as managing devices, memory management, or interrupt handling. The command `ps -ef` can be used to display the kernel threads on a running Linux system. Examining the output of this command will show the kernel thread `kthreadd` (with `pid` = 2), which serves as the parent of all other kernel threads.

Many applications can also take advantage of multiple threads, including basic sorting, trees, and graph algorithms. In addition, programmers who must solve contemporary CPU-intensive problems in data mining, graphics, and artificial intelligence can leverage the power of modern multicore systems by designing solutions that run in parallel.

Benefits

The benefits of multithreaded programming can be broken down into four major categories:

1. **Responsiveness.** Multithreading an interactive application may allow a program to continue running even if part of it is blocked or is performing a lengthy operation, thereby increasing responsiveness to the user. This quality is especially useful in designing user interfaces. For instance, consider what happens when a user clicks a button that results in the performance of a time-consuming operation. A single-threaded application would be unresponsive to the user until the operation had been

completed. In contrast, if the time-consuming operation is performed in a separate, asynchronous thread, the application remains responsive to the user.

2. **Resource sharing.** Processes can share resources only through techniques such as shared memory and message passing. Such techniques must be explicitly arranged by the programmer. However, threads share the memory and the resources of the process to which they belong by default. The benefit of sharing code and data is that it allows an application to have several different threads of activity within the same address space.
3. **Economy.** Allocating memory and resources for process creation is costly. Because threads share the resources of the process to which they belong, it is more economical to create and context-switch threads. Empirically gauging the difference in overhead can be difficult, but in general thread creation consumes less time and memory than process creation. Additionally, context switching is typically faster between threads than between processes.
4. **Scalability.** The benefits of multithreading can be even greater in a multiprocessor architecture, where threads may be running in parallel on different processing cores. A single-threaded process can run on only one processor, regardless how many are available. We explore this issue further in the following section.

PARTICIPATION ACTIVITY 3.1.1: Section review questions.

1) What is the relationship between threads and processes?

- A process consists of one or more threads.
- A thread consists of one or more processes.
- A thread is a separate entity than a process.

2) Which of the following is not shared by threads?

- Code
- Stack
- Data

Section glossary

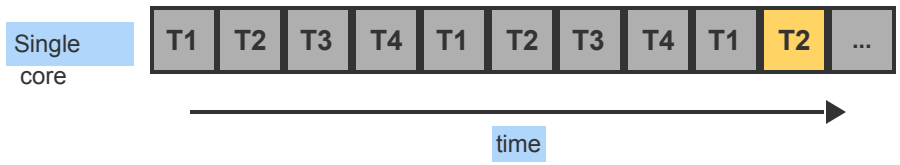
single-threaded: A process or program that has only one thread of control (and so executes on only one core at a time).

multithreaded: A term describing a process or program with multiple threads of control, allowing multiple simultaneous execution points.

3.2 Multicore programming

Earlier in the history of computer design, in response to the need for more computing performance, single-CPU systems evolved into multi-CPU systems. A later, yet similar, trend in system design is to place multiple computing cores on a single processing chip where each core appears as a separate CPU to the operating system (Section Multiprocessor systems). We refer to such systems as ***multicore***, and multithreaded programming provides a mechanism for more efficient use of these multiple computing cores and improved concurrency. Consider an application with four threads. On a system with a single computing core, concurrency merely means that the execution of the threads will be interleaved over time (animation titled Concurrent execution on a single-core system), because the processing core is capable of executing only one thread at a time. On a system with multiple cores, however, concurrency means that some threads can run in parallel, because the system can assign a separate thread to each core (animation titled Parallel execution on a dual-core system).

PARTICIPATION ACTIVITY 3.2.1: Concurrent execution on a single-core system.



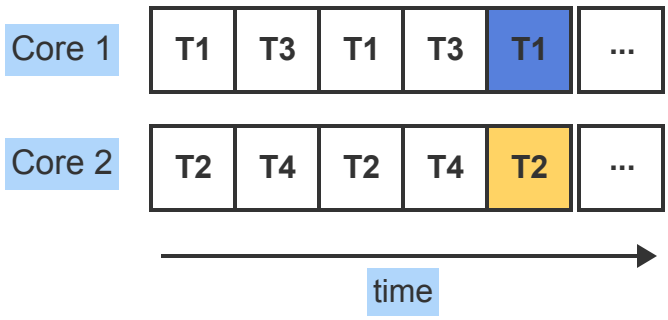
Animation content:

Step 1: Concurrency means all threads make progress on a single-core system as each thread gets to run for a short period of time on the single processing core. Animation shows tasks being handled on a single-core processor over time.

Animation captions:

1. Concurrency means all threads make progress on a single-core system as each thread gets to run for a short period of time on the single processing core.

PARTICIPATION ACTIVITY 3.2.2: Parallel execution on a dual-core system.



Animation content:

Step 1: Parallelism allows two threads to run at the same time as each thread runs on a separate processing core.

Animation captions:

1. Parallelism allows two threads to run at the same time as each thread runs on a separate processing core.

Notice the distinction between ***concurrency*** and ***parallelism*** in this discussion. A concurrent system supports more than one task by allowing all the tasks to make progress. In contrast, a parallel system can perform more than one task simultaneously. Thus, it is possible to have concurrency without parallelism. Before the advent of multiprocessor and multicore architectures, most computer systems had only a single processor, and CPU schedulers were designed to provide the illusion of parallelism by rapidly

switching between processes, thereby allowing each process to make progress. Such processes were running concurrently, but not in parallel.

Programming challenges

The trend toward multicore systems continues to place pressure on system designers and application programmers to make better use of the multiple computing cores. Designers of operating systems must write scheduling algorithms that use multiple processing cores to allow the parallel execution shown in the animation titled Parallel execution on a dual-core system. For application programmers, the challenge is to modify existing programs as well as design new programs that are multithreaded.

In general, five areas present challenges in programming for multicore systems:

1. **Identifying tasks.** This involves examining applications to find areas that can be divided into separate, concurrent tasks. Ideally, tasks are independent of one another and thus can run in parallel on individual cores.
2. **Balance.** While identifying tasks that can run in parallel, programmers must also ensure that the tasks perform equal work of equal value. In some instances, a certain task may not contribute as much value to the overall process as other tasks. Using a separate execution core to run that task may not be worth the cost.
3. **Data splitting.** Just as applications are divided into separate tasks, the data accessed and manipulated by the tasks must be divided to run on separate cores.
4. **Data dependency.** The data accessed by the tasks must be examined for dependencies between two or more tasks. When one task depends on data from another, programmers must ensure that the execution of the tasks is synchronized to accommodate the data dependency. We examine such strategies in the chapter Synchronization Tools.
5. **Testing and debugging.** When a program is running in parallel on multiple cores, many different execution paths are possible. Testing and debugging such concurrent programs is inherently more difficult than testing and debugging single-threaded applications.

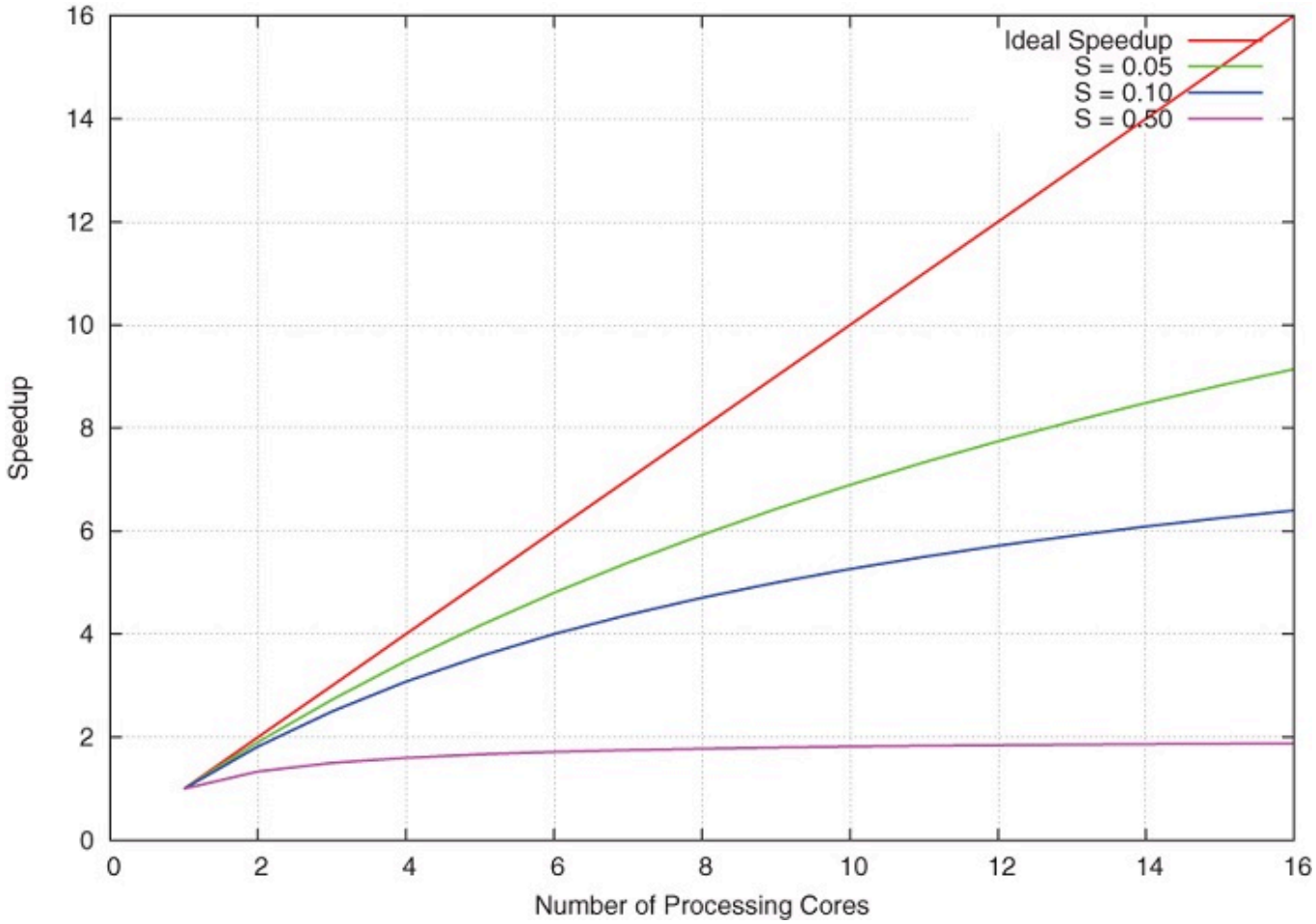
Because of these challenges, many software developers argue that the advent of multicore systems will require an entirely new approach to designing software systems in the future. (Similarly, many computer science educators believe that software development must be taught with increased emphasis on parallel programming.)

Amdahl's law

Amdahl's Law is a formula that identifies potential performance gains from adding additional computing cores to an application that has both serial (nonparallel) and parallel components. If S is the portion of the application that must be performed serially on a system with N processing cores, the formula appears as follows:

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

As an example, assume we have an application that is 75 percent parallel and 25 percent serial. If we run this application on a system with two processing cores, we can get a speedup of 1.6 times. If we add two additional cores (for a total of four), the speedup is 2.28 times. Below is a graph illustrating Amdahl's Law in several different scenarios.



One interesting fact about Amdahl's Law is that as N approaches infinity, the speedup converges to $1/S$. For example, if 50 percent of an application is performed serially, the maximum speedup is 2.0 times, regardless of the number of processing cores we add. This is the fundamental principle behind Amdahl's Law: the serial portion of an application can have a disproportionate effect on the performance we gain by adding additional computing cores.

Types of parallelism

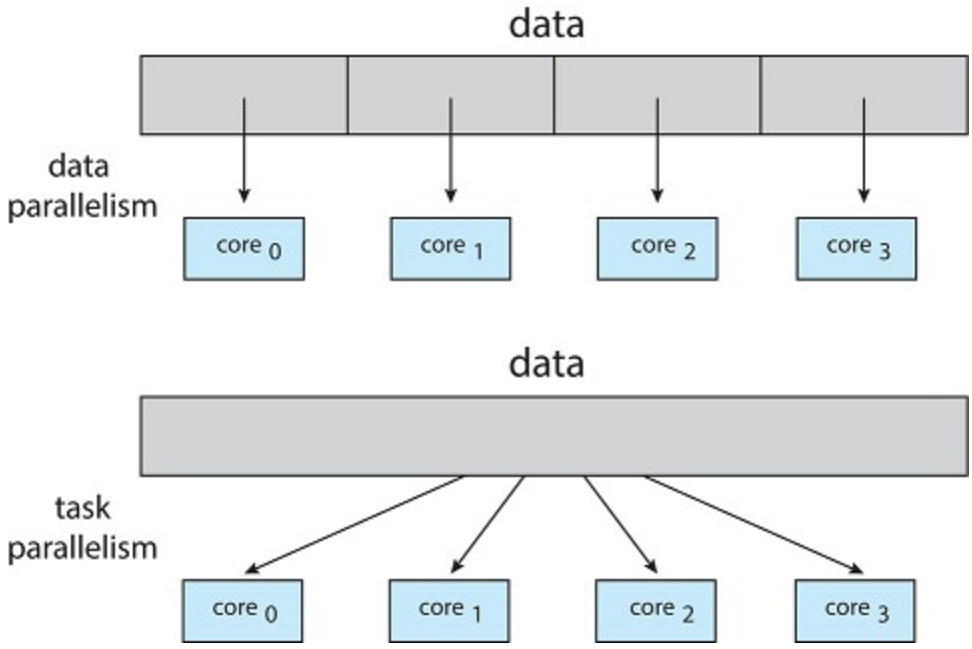
In general, there are two types of parallelism: data parallelism and task parallelism. ***Data parallelism*** focuses on distributing subsets of the same data across multiple computing cores and performing the

same operation on each core. Consider, for example, summing the contents of an array of size N. On a single-core system, one thread would simply sum the elements $[0]$ … $[N-1]$. On a dual-core system, however, thread A, running on core 0, could sum the elements $[0]$ … $[N/2-1]$ while thread B, running on core 1, could sum the elements $[N/2]$ … $[N-1]$. The two threads would be running in parallel on separate computing cores.

Task parallelism involves distributing not data but tasks (threads) across multiple computing cores. Each thread is performing a unique operation. Different threads may be operating on the same data, or they may be operating on different data. Consider again our example above. In contrast to that situation, an example of task parallelism might involve two threads, each performing a unique statistical operation on the array of elements. The threads again are operating in parallel on separate computing cores, but each is performing a unique operation.

Fundamentally, then, data parallelism involves the distribution of data across multiple cores, and task parallelism involves the distribution of tasks across multiple cores, as shown in Figure 3.2.1. However, data and task parallelism are not mutually exclusive, and an application may in fact use a hybrid of these two strategies.

Figure 3.2.1: Data and task parallelism.



PARTICIPATION ACTIVITY 3.2.3: Section review questions.

1) _____ system allows more than one task to run simultaneously.

- Concurrent
- Parallel
- Multithreaded

2) _____ involves distributing tasks across multiple computing cores.

- Concurrency
- Task parallelism
- Data parallelism

3) According to Amdahl's Law, what is the maximum speedup gain on a system with 4 processing cores for an application that is 20% serial?

- 1.18
- 4
- 2.50

Section glossary

multicore: Multiple processing cores within the same CPU chip or within a single system.

data parallelism: A computing method that distributes subsets of the same data across multiple cores and performs the same operation on each core.

task parallelism: A computing method that distributes tasks (threads) across multiple computing cores, with each task is performing a unique operation.

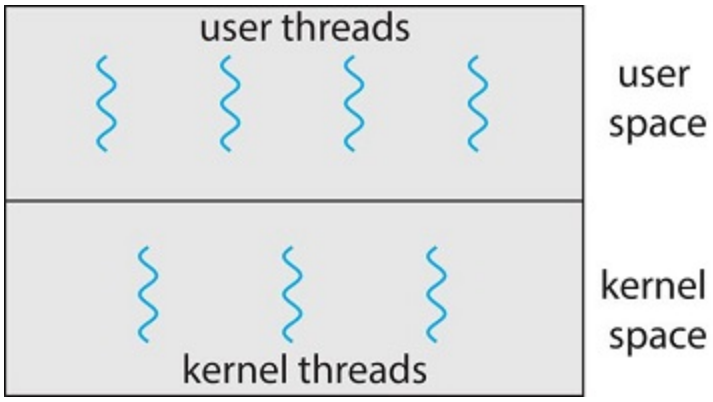
3.3 Multithreading models

Our discussion so far has treated threads in a generic sense. However, support for threads may be provided either at the user level, for ***user threads***, or by the kernel, for ***kernel threads***. User threads are supported above the kernel and are managed without kernel support, whereas kernel threads are

supported and managed directly by the operating system. Virtually all contemporary operating systems—including Windows, Linux, and macOS—support kernel threads.

Ultimately, a relationship must exist between user threads and kernel threads, as illustrated in Figure 3.3.1. In this section, we look at three common ways of establishing such a relationship: the many-to-one model, the one-to-one model, and the many-to-many model.

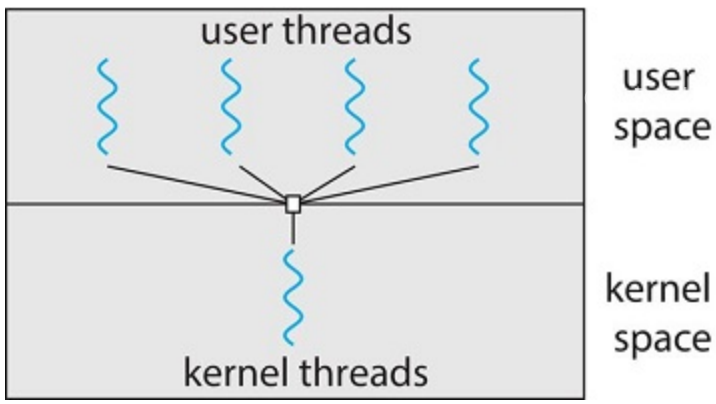
Figure 3.3.1: User and kernel threads.



Many-to-one model

The many-to-one model (Figure 3.3.2) maps many user-level threads to one kernel thread. Thread management is done by the thread library in user space, so it is efficient (we discuss thread libraries in Section 3.4). However, the entire process will block if a thread makes a blocking system call. Also, because only one thread can access the kernel at a time, multiple threads are unable to run in parallel on multicore systems. ***Green threads***—a thread library available for Solaris systems and adopted in early versions of Java—used the many-to-one model. However, very few systems continue to use the model because of its inability to take advantage of multiple processing cores, which have now become standard on most computer systems.

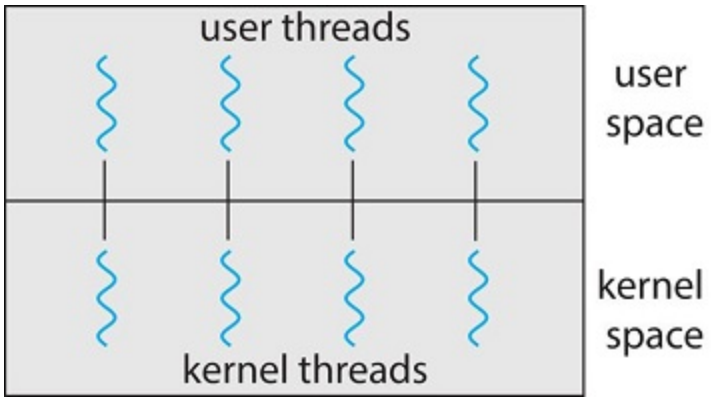
Figure 3.3.2: Many-to-one model.



One-to-one model

The one-to-one model (Figure 3.3.3) maps each user thread to a kernel thread. It provides more concurrency than the many-to-one model by allowing another thread to run when a thread makes a blocking system call. It also allows multiple threads to run in parallel on multiprocessors. The only drawback to this model is that creating a user thread requires creating the corresponding kernel thread, and a large number of kernel threads may burden the performance of a system. Linux, along with the family of Windows operating systems, implement the one-to-one model.

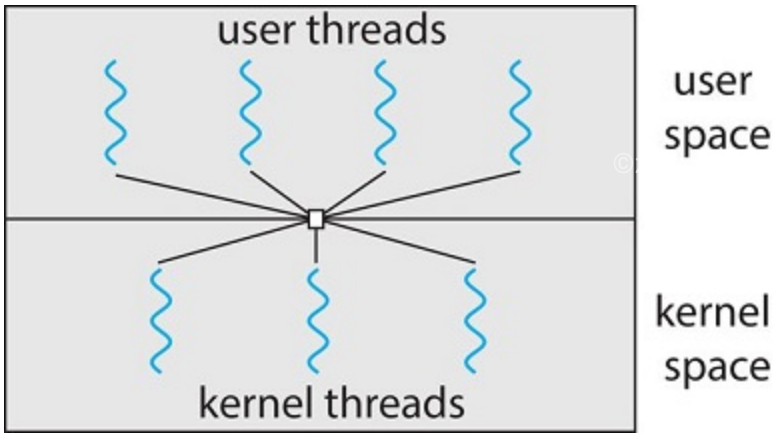
Figure 3.3.3: One-to-one model.



Many-to-many model

The many-to-many model (Figure 3.3.4) multiplexes many user-level threads to a smaller or equal number of kernel threads. The number of kernel threads may be specific to either a particular application or a particular machine (an application may be allocated more kernel threads on a system with eight processing cores than a system with four cores).

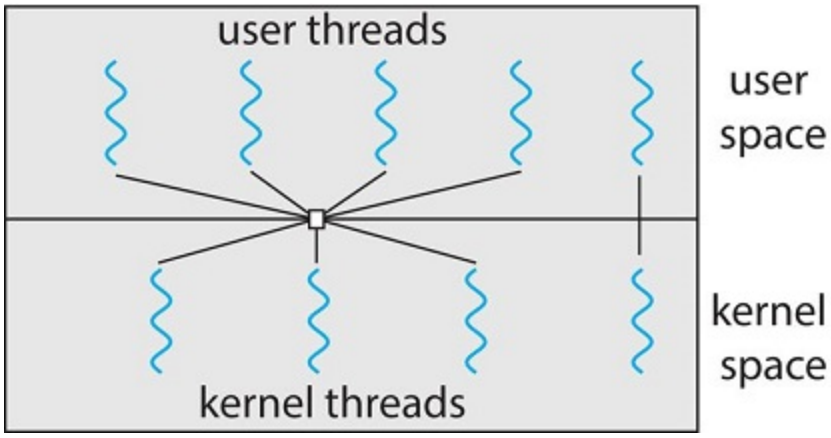
Figure 3.3.4: Many-to-many model.



Let's consider the effect of this design on concurrency. Whereas the many-to-one model allows the developer to create as many user threads as she wishes, it does not result in parallelism, because the kernel can schedule only one kernel thread at a time. The one-to-one model allows greater concurrency, but the developer has to be careful not to create too many threads within an application. (In fact, on some systems, she may be limited in the number of threads she can create.) The many-to-many model suffers from neither of these shortcomings: developers can create as many user threads as necessary, and the corresponding kernel threads can run in parallel on a multiprocessor. Also, when a thread performs a blocking system call, the kernel can schedule another thread for execution.

One variation on the many-to-many model still multiplexes many user-level threads to a smaller or equal number of kernel threads but also allows a user-level thread to be bound to a kernel thread. This variation is sometimes referred to as the ***two-level model*** (Figure 3.3.5).

Figure 3.3.5: Two-level model.



Although the many-to-many model appears to be the most flexible of the models discussed, in practice it is difficult to implement. In addition, with an increasing number of processing cores appearing on most systems, limiting the number of kernel threads has become less important. As a result, most operating systems now use the one-to-one model. However, as we shall see in Section 3.5, some contemporary concurrency libraries have developers identify tasks that are then mapped to threads using the many-to-many model.

PARTICIPATION ACTIVITY 3.3.1: Section review questions.

1) The _____ multithreading model multiplexes many user-level threads to a smaller or equal number of kernel threads.

- many-to-one
- one-to-one
- many-to-many

2) The _____ multithreading model is used by both Linux and Windows.

- many-to-one
- one-to-one
- many-to-many

Section glossary

user thread: A thread running in user mode.

kernel threads: Threads running in kernel mode.

3.4 Thread libraries

A ***thread library*** provides the programmer with an API for creating and managing threads. There are two primary ways of implementing a thread library. The first approach is to provide a library entirely in user space with no kernel support. All code and data structures for the library exist in user space. This means that invoking a function in the library results in a local function call in user space and not a system call.

The second approach is to implement a kernel-level library supported directly by the operating system. In this case, code and data structures for the library exist in kernel space. Invoking a function in the API for the library typically results in a system call to the kernel.

Three main thread libraries are in use today: POSIX Pthreads, Windows, and Java. Pthreads, the threads extension of the POSIX standard, may be provided as either a user-level or a kernel-level library. The Windows thread library is a kernel-level library available on Windows systems. The Java thread API allows threads to be created and managed directly in Java programs. However, because in most instances the JVM is running on top of a host operating system, the Java thread API is generally implemented using a thread library available on the host system. This means that on Windows systems, Java threads are typically implemented using the Windows API; UNIX, Linux, and macOS systems typically use Pthreads.

For POSIX and Windows threading, any data declared globally—that is, declared outside of any function—are shared among all threads belonging to the same process. Because Java has no equivalent notion of global data, access to shared data must be explicitly arranged between threads.

In the remainder of this section, we describe basic thread creation using these three thread libraries. As an illustrative example, we design a multithreaded program that performs the summation of a non-negative integer in a separate thread using the well-known summation function:

$$sum = \sum_{t=1}^{N} i$$

For example, if N were 5, this function would represent the summation of integers from 1 to 5, which is 15. Each of the three programs will be run with the upper bounds of the summation entered on the command line. Thus, if the user enters 8, the summation of the integer values from 1 to 8 will be output.

Before we proceed with our examples of thread creation, we introduce two general strategies for creating multiple threads: ***asynchronous threading*** and ***synchronous threading***. With asynchronous threading, once the parent creates a child thread, the parent resumes its execution, so that the parent and child execute concurrently and independently of one another. Because the threads are independent, there is typically little data sharing between them. Asynchronous threading is the strategy used in the multithreaded server illustrated in Figure 3.1.2 and is also commonly used for designing responsive user interfaces.

Synchronous threading occurs when the parent thread creates one or more children and then must wait for all of its children to terminate before it resumes. Here, the threads created by the parent perform work concurrently, but the parent cannot continue until this work has been completed. Once each thread has finished its work, it terminates and joins with its parent. Only after all of the children have joined can the parent resume execution. Typically, synchronous threading involves significant data sharing among threads. For example, the parent thread may combine the results calculated by its various children. All of the following examples use synchronous threading.

Pthreads

Pthreads refers to the POSIX standard (IEEE 1003.1c) defining an API for thread creation and synchronization. This is a ***specification*** for thread behavior, not an ***implementation***. Operating-system designers may implement the specification in any way they wish. Numerous systems implement the Pthreads specification; most are UNIX-type systems, including Linux and macOS. Although Windows doesn't support Pthreads natively, some third-party implementations for Windows are available.

The C program shown in Figure 3.4.1 demonstrates the basic Pthreads API for constructing a multithreaded program that calculates the summation of a non-negative integer in a separate thread. In a Pthreads program, separate threads begin execution in a specified function. In Figure 3.4.1, this is the `runner()` function. When this program begins, a single thread of control begins in `main()`. After some initialization, `main()` creates a second thread that begins control in the `runner()` function. Both threads share the global data `sum`.

Figure 3.4.1: Multithreaded C program using the Pthreads API.

```
#include <pthread.h>
#include <stdio.h>

#include <stdlib.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
  pthread_t tid;  /* the thread identifier */
  pthread_attr_t attr; /* set of thread attributes */

  /* set the default attributes of the thread */
  pthread_attr_init(&attr);
  /* create the thread */
  pthread_create(&tid, &attr, runner, argv[1]);
  /* wait for the thread to exit */
  pthread_join(tid,NULL);

  printf("sum = %d\n",sum);
}

/* The thread will execute in this function */
void *runner(void *param)
{
  int i, upper = atoi(param);
  sum = 0;

  for (i = 1; i <= upper; i++)
    sum += i;

  pthread_exit(0);
}
```


Let's look more closely at this program. All Pthreads programs must include the `pthread.h` header file. The statement `pthread_t tid` declares the identifier for the thread we will create. Each thread has a set of attributes, including stack size and scheduling information. The `pthread_attr_t attr` declaration represents the attributes for the thread. We set the attributes in the function call `pthread_attr_init(&attr)`. Because we did not explicitly set any attributes, we use the default attributes provided. (In the chapter CPU Scheduling, we discuss some of the scheduling attributes provided by the Pthreads API.) A separate thread is created with the `pthread_create()` function call. In addition to passing the thread identifier and the attributes for the thread, we also pass the name of the function where the new thread will begin execution—in this case, the `runner()` function. Last, we pass the integer parameter that was provided on the command line, `argv[1]`.

At this point, the program has two threads: the initial (or parent) thread in `main()` and the summation (or child) thread performing the summation operation in the `runner()` function. This program follows the thread create/join strategy, whereby after creating the summation thread, the parent thread will wait for it to terminate by calling the `pthread_join()` function. The summation thread will terminate when it calls

the function `pthread_exit()`. Once the summation thread has returned, the parent thread will output the value of the shared data `sum`.

This example program creates only a single thread. With the growing dominance of multicore systems, writing programs containing several threads has become increasingly common. A simple method for waiting on several threads using the `pthread_join()` function is to enclose the operation within a simple `for` loop. For example, you can join on ten threads using the Pthread code shown in Figure 3.4.2.

Figure 3.4.2: Pthread code for joining ten threads.

```
#define NUM THREADS 10

/* an array of threads to be joined upon */
pthread_t workers[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++)
  pthread_join(workers[i], NULL);
```

PARTICIPATION ACTIVITY 3.4.1: Pthreads functions for managing threads.

Provide the ordering of the Pthreads functions that (1) initialize a pthread attribute, (2) create a thread that runs the function named work(), and (3) wait for the thread to complete.

Assume you have the following variables available:

pthread_t thread_id;

pthread_attr_t thread_attr;

How to use this tool

Unused blocks

pthread_attr_init(&thread_attr);

pthread_join(thread_id,NULL);

pthread_create(&thread_id, &thread_attr, work, NULL);

Proof Reset

Move blocks here

0 of 3 blocks correct

PARTICIPATION ACTIVITY 3.4.2: Parameters to pthreads_create() function.

Provide the necessary code that creates a Pthread that will run the function named compute(), passing the parameter argv[2] to the compute() function. Assume you have the following variables available:

pthread_t thread_id;

pthread_attr_t thread_attr;

How to use this tool

Unused blocks

- &thread_id,
-);
- argv[2]
- compute,
- pthread_create(
- &thread_attr,

Proof Reset

Move blocks here

0 of 6 blocks correct

Windows threads

The technique for creating threads using the Windows thread library is similar to the Pthreads technique in several ways. We illustrate the Windows thread API in the C program shown in Figure 3.4.3. Notice that we must include the `windows.h` header file when using the Windows API.

Figure 3.4.3: Multithreaded C program using the Windows API.

```
#include <windows.h>
#include <stdio.h>
```

```
DWORD Sum; /* data is shared by the thread(s) */

/* The thread will execute in this function */
DWORD WINAPI Summation(LPVOID Param)
{
  DWORD Upper = *(DWORD*)Param;
  for (DWORD i = 1; i <= Upper; i++)
    Sum += i;
  return 0;
}

int main(int argc, char *argv[])
{
  DWORD ThreadId;
  HANDLE ThreadHandle;
  int Param;

  Param = atoi(argv[1]);
  /* create the thread */
  ThreadHandle = CreateThread(
    NULL, /* default security attributes */
    0, /* default stack size */
    Summation, /* thread function */
    &Param, /* parameter to thread function */
    0, /* default creation flags */
    &ThreadId); /* returns the thread identifier */

   /* now wait for the thread to finish */
 WaitForSingleObject(ThreadHandle,INFINITE);

  /* close the thread handle */
  CloseHandle(ThreadHandle);

  printf("sum = %d\n",Sum);
}
```


Just as in the Pthreads version shown in Figure 3.4.1, data shared by the separate threads—in this case, **`Sum`**—are declared globally (the **`DWORD`** data type is an unsigned 32-bit integer). We also define the **`Summation()`** function that is to be performed in a separate thread. This function is passed a pointer to a **`void`**, which Windows defines as LPVOID. The thread performing this function sets the global data **`Sum`** to the value of the summation from 0 to the parameter passed to **`Summation()`**.

Threads are created in the Windows API using the **`CreateThread()`** function, and—just as in Pthreads—a set of attributes for the thread is passed to this function. These attributes include security information, the size of the stack, and a flag that can be set to indicate if the thread is to start in a suspended state. In this program, we use the default values for these attributes. (The default values do not initially set the thread to a suspended state and instead make it eligible to be run by the CPU scheduler.) Once the summation thread is created, the parent must wait for it to complete before outputting the value of **`Sum`**, as the value is set by the summation thread. Recall that the Pthread program (Figure 3.4.1) had the parent thread wait for the summation thread using the **`pthread_join()`** statement. We perform the equivalent of this in the Windows API using the **`WaitForSingleObject()`** function, which causes the creating thread to block until the summation thread has exited.

In situations that require waiting for multiple threads to complete, the `WaitForMultipleObjects()` function is used. This function is passed four parameters:

1. The number of objects to wait for
2. A pointer to the array of objects
3. A flag indicating whether all objects have been signaled
4. A timeout duration (or `INFINITE`)

For example, if `THandles` is an array of thread `HANDLE` objects of size `N`, the parent thread can wait for all its child threads to complete with this statement:

```
WaitForMultipleObjects(N, THandles, TRUE, INFINITE);
```

Java threads

Threads are the fundamental model of program execution in a Java program, and the Java language and its API provide a rich set of features for the creation and management of threads. All Java programs comprise at least a single thread of control—even a simple Java program consisting of only a `main()` method runs as a single thread in the JVM. Java threads are available on any system that provides a JVM, including Windows, Linux, and macOS. The Java thread API is available for Android applications as well.

There are two techniques for explicitly creating threads in a Java program. One approach is to create a new class that is derived from the `Thread` class and to override its `run()` method. An alternative—and more commonly used—technique is to define a class that implements the `Runnable` interface. This interface defines a single abstract method with the signature `public void run()`. The code in the `run()` method of a class that implements `Runnable` is what executes in a separate thread. An example is shown below:

```
class Task implements Runnable
{
  public void run() {
    System.out.println("I am a thread.");
  }
}
```

Lambda expressions in java

Beginning with Version 1.8 of the language, Java introduced Lambda expressions, which allow a much cleaner syntax for creating threads. Rather than defining a separate class that implements `Runnable`, a Lambda expression can be used instead:

```
Runnable task = () -> {
   System.out.println("I am a thread.");
};

Thread worker = new Thread(task);
worker.start();
```

> Lambda expressions—as well as similar functions known as ***closures***—are a prominent feature of functional programming languages and have been available in several nonfunctional languages as well including Python, C++, and C#. As we shall see in later examples in this chapter, Lamdba expressions often provide a simple syntax for developing parallel applications.

Thread creation in Java involves creating a `Thread` object and passing it an instance of a class that implements `Runnable`, followed by invoking the `start()` method on the `Thread` object. This appears in the following example:

```
Thread worker = new Thread(new Task());
worker.start();
```

Invoking the `start()` method for the new `Thread` object does two things:

1. It allocates memory and initializes a new thread in the JVM.
2. It calls the `run()` method, making the thread eligible to be run by the JVM. (Note again that we never call the `run()` method directly. Rather, we call the `start()` method, and it calls the `run()` method on our behalf.)

Recall that the parent threads in the Pthreads and Windows libraries use `pthread_join()` and `WaitForSingleObject()` (respectively) to wait for the summation threads to finish before proceeding. The `join()` method in Java provides similar functionality. (Notice that `join()` can throw an `InterruptedException`, which we choose to ignore.)

```
try {
  worker.join();
}
catch (InterruptedException ie) { }
```

If the parent must wait for several threads to finish, the `join()` method can be enclosed in a `for` loop similar to that shown for Pthreads in Figure 3.4.2.

Java executor framework

Java has supported thread creation using the approach we have described thus far since its origins. However, beginning with Version 1.5 and its API, Java introduced several new concurrency features that provide developers with much greater control over thread creation and communication. These tools are available in the `java.util.concurrent` package.

Rather than explicitly creating `Thread` objects, thread creation is instead organized around the `Executor` interface:

```
public interface Executor
{
```

```
  void execute(Runnable command);
}
```

Classes implementing this interface must define the **`execute()`** method, which is passed a **`Runnable`** object. For Java developers, this means using the **`Executor`** rather than creating a separate **`Thread`** object and invoking its **`start()`** method. The **`Executor`** is used as follows:

```
Executor service = new Executor;
service.execute(new Task());
```


The **`Executor`** framework is based on the producer-consumer model; tasks implementing the **`Runnable`** interface are produced, and the threads that execute these tasks consume them. The advantage of this approach is that it not only divides thread creation from execution but also provides a mechanism for communication between concurrent tasks.

Data sharing between threads belonging to the same process occurs easily in Windows and Pthreads, since shared data are simply declared globally. As a pure object-oriented language, Java has no such notion of global data. We can pass parameters to a class that implements **`Runnable`**, but Java threads cannot return results. To address this need, the **`java.util.concurrent`** package additionally defines the **`Callable`** interface, which behaves similarly to **`Runnable`** except that a result can be returned. Results returned from **`Callable`** tasks are known as **`Future`** objects. A result can be retrieved from the **`get()`** method defined in the **`Future`** interface. The program shown in Figure 3.4.4 illustrates the summation program using these Java features.

Figure 3.4.4: Illustration of Java Executor framework API.

```
import java.util.concurrent.*;

class Summation implements Callable<Integer>
{
  private int upper;
  public Summation(int upper) {
    this.upper = upper;
  }

  /* The thread will execute in this method */
  public Integer call() {
    int sum = 0;
    for (int i = 1; i <= upper; i++)
      sum += i;

    return new Integer(sum);
  }
}

public class Driver
{
 public static void main(String[] args) {
   int upper = Integer.parseInt(args[0]);
```

```
    ExecutorService pool = Executors.newSingleThreadExecutor();
    Future<Integer> result = pool.submit(new Summation(upper));

    try {
        System.out.println("sum = " + result.get());
    } catch (InterruptedException | ExecutionException ie) { }
  }
}
```

The `Summation` class implements the `Callable` interface, which specifies the method `V call()`—it is the code in this `call()` method that is executed in a separate thread. To execute this code, we create a `newSingleThreadExecutor` object (provided as a static method in the `Executors` class), which is of type `ExecutorService`, and pass it a `Callable` task using its `submit()` method. (The primary difference between the `execute()` and `submit()` methods is that the former returns no result, whereas the latter returns a result as a `Future`.) Once we submit the `callable` task to the thread, we wait for its result by calling the `get()` method of the `Future` object it returns.

It is quite easy to notice at first that this model of thread creation appears more complicated than simply creating a thread and joining on its termination. However, incurring this modest degree of complication confers benefits. As we have seen, using `Callable` and `Future` allows for threads to return results. Additionally, this approach separates the creation of threads from the results they produce: rather than waiting for a thread to terminate before retrieving results, the parent instead only waits for the results to become available. Finally, as we shall see in Section Thread Pools, this framework can be combined with other features to create robust tools for managing a large number of threads.

PARTICIPATION ACTIVITY | 3.4.3: Creating threads in Java. | Full screen

Create a Java thread that first outputs the message "I am a thread." followed by the main thread which outputs the message "Finished." Not all code blocks may need to be used.

How to use this tool

Unused

```
Thread worker = new Thread(new Worker());
```

```
public void run() {
      System.out.println("I am a thread.\n");
}
```

```
class Worker implements Runnable
{
```

```
System.out.println("Finished.\n");
```

```
worker.run();
```

```
worker.start();
```

```
try {
   worker.join();
}
catch (InterruptedException ie) { }
```

JavaThreadPractice.java Load default template...

```
public class JavaThreadPractice {
   public static void main (String [] args) {
   }
}
}
```

Check

PARTICIPATION ACTIVITY 3.4.4: Section review questions.

1) A ____ provides an API for creating and managing threads.

   - multicore system
   - thread library
   - multithreading model

2) In Pthreads and Windows threads any data declared _____ are shared by all threads belonging to the same process.

   - locally
   - globally
   - using shared memory

3) What does the third parameter passed to the `pthread_create()` function specify?

- The parameter being passed to the thread being created.
- The thread identifier
- The name of the function that will be run by the thread being created.

4) What is the Windows function call that causes a parent thread to wait for a child thread to finish?

- `WaitForSingleObject()`
- `pthread_join()`
- `join()`

5) What is the type of parameter that is passed to the Java Thread() constructor?

- The name of the method that is to run in a separate thread.
- The name of a class that extends the Thread object.
- The name of a class the implements the Runnable interface.

Section glossary

thread library: A programming library that provides programmers with an API for creating and managing threads.

asynchronous threading: Threading in which a parent creates a child thread and then resumes execution, so that the parent and child execute concurrently and independently of one another.

synchronous threading: Threading in which a parent thread creating one or more child threads waits for them to terminate before it resumes.

Pthreads: The POSIX standard (IEEE 1003.1c) defining an API for thread creation and synchronization (a specification for thread behavior, not an implementation).

closure: In functional programming languages, a construct to provide a simple syntax for parallel applications.

3.5 Implicit threading

With the continued growth of multicore processing, applications containing hundreds—or even thousands—of threads are looming on the horizon. Designing such applications is not a trivial undertaking: programmers must address not only the challenges outlined in Section 3.2 but additional difficulties as well. These difficulties, which relate to program correctness, are covered in the chapter Synchronization Tools and chapter Deadlocks.

The JVM and the host operating system

The JVM is typically implemented on top of a host operating system (see Figure Missing Reference : The Java virtual machine.). This setup allows the JVM to hide the implementation details of the underlying operating system and to provide a consistent, abstract environment that allows Java programs to operate on any platform that supports a JVM. The specification for the JVM does not indicate how Java threads are to be mapped to the underlying operating system, instead leaving that decision to the particular implementation of the JVM. For example, the Windows operating system uses the one-to-one model; therefore, each Java thread for a JVM running on Windows maps to a kernel thread. In addition, there may be a relationship between the Java thread library and the thread library on the host operating system. For example, implementations of a JVM for the Windows family of operating systems might use the Windows API when creating Java threads; Linux and macOS systems might use the Pthreads API.

One way to address these difficulties and better support the design of concurrent and parallel applications is to transfer the creation and management of threading from application developers to compilers and run-time libraries. This strategy, termed ***implicit threading***, is an increasingly popular trend. In this section, we explore four alternative approaches to designing applications that can take advantage of multicore processors through implicit threading. As we shall see, these strategies generally require application developers to identify ***tasks***—not threads—that can run in parallel. A task is usually written as a function, which the run-time library then maps to a separate thread, typically using the many-to-many model (Section Many-to-many model). The advantage of this approach is that developers only need to identify parallel tasks, and the libraries determine the specific details of thread creation and management.

Thread pools

In Section 3.1, we described a multithreaded web server. In this situation, whenever the server receives a request, it creates a separate thread to service the request. Whereas creating a separate thread is certainly superior to creating a separate process, a multithreaded server nonetheless has potential problems. The first issue concerns the amount of time required to create the thread, together with the fact that the thread will be discarded once it has completed its work. The second issue is more troublesome. If we allow each concurrent request to be serviced in a new thread, we have not placed a bound on the number of threads concurrently active in the system. Unlimited threads could exhaust system resources, such as CPU time or memory. One solution to this problem is to use a ***thread pool***.

Android thread pools

> In Section Android RPC, we covered RPCs in the Android operating system. You may recall from that section that Android uses the Android Interface Definition Language (AIDL), a tool that specifies the remote interface that clients interact with on the server. AIDL also provides a thread pool. A remote service using the thread pool can handle multiple concurrent requests, servicing each request using a separate thread from the pool.

The general idea behind a thread pool is to create a number of threads at start-up and place them into a pool, where they sit and wait for work. When a server receives a request, rather than creating a thread, it instead submits the request to the thread pool and resumes waiting for additional requests. If there is an available thread in the pool, it is awakened, and the request is serviced immediately. If the pool contains no available thread, the task is queued until one becomes free. Once a thread completes its service, it returns to the pool and awaits more work. Thread pools work well when the tasks submitted to the pool can be executed asynchronously.

Thread pools offer these benefits:

1. Servicing a request with an existing thread is often faster than waiting to create a thread.
2. A thread pool limits the number of threads that exist at any one point. This is particularly important on systems that cannot support a large number of concurrent threads.
3. Separating the task to be performed from the mechanics of creating the task allows us to use different strategies for running the task. For example, the task could be scheduled to execute after a time delay or to execute periodically.

The number of threads in the pool can be set heuristically based on factors such as the number of CPUs in the system, the amount of physical memory, and the expected number of concurrent client requests. More sophisticated thread-pool architectures can dynamically adjust the number of threads in the pool according to usage patterns. Such architectures provide the further benefit of having a smaller pool—thereby consuming less memory—when the load on the system is low. We discuss one such architecture, Apple's Grand Central Dispatch, later in this section.

PARTICIPATION ACTIVITY 3.5.1: Mid-section review question.

A _____ uses an existing thread — rather than creating a new one — to complete a task.

Check Show answer

The Windows API provides several functions related to thread pools. Using the thread pool API is similar to creating a thread with the `Thread_Create()` function, as described in Section Windows threads. Here, a function that is to run as a separate thread is defined. Such a function may appear as follows:

```
DWORD WINAPI PoolFunction(PVOID Param) {
   /* this function runs as a separate thread. */
}
```

A pointer to `PoolFunction()` is passed to one of the functions in the thread pool API, and a thread from the pool executes this function. One such member in the thread pool API is the `QueueUserWorkItem()` function, which is passed three parameters:

- `LPTHREAD_START_ROUTINE Function`—a pointer to the function that is to run as a separate thread
- `PVOID Param`—the parameter passed to `Function`
- `ULONG Flags`—flags indicating how the thread pool is to create and manage execution of the thread

An example of invoking a function is the following:

```
QueueUserWorkItem(&PoolFunction, NULL, 0);
```

This causes a thread from the thread pool to invoke `PoolFunction()` on behalf of the programmer. In this instance, we pass no parameters to `PoolFunction()`. Because we specify `0` as a flag, we provide the thread pool with no special instructions for thread creation.

Other members in the Windows thread pool API include utilities that invoke functions at periodic intervals or when an asynchronous I/O request completes.

Java thread pools

The `java.util.concurrent` package includes an API for several varieties of thread-pool architectures. Here, we focus on the following three models:

1. Single thread executor—`newSingleThreadExecutor()`—creates a pool of size 1.
2. Fixed thread executor—`newFixedThreadPool(int size)`—creates a thread pool with a specified number of threads.
3. Cached thread executor—`newCachedThreadPool()`—creates an unbounded thread pool, reusing threads in many instances.

We have, in fact, already seen the use of a Java thread pool in Section Java threads, where we created a `newSingleThreadExecutor` in the program example shown in Figure 3.4.4. In that section, we noted that the Java executor framework can be used to construct more robust threading tools. We now describe how it can be used to create thread pools.

A thread pool is created using one of the factory methods in the `Executors` class:

- `static ExecutorService newSingleThreadExecutor()`
- `static ExecutorService newFixedThreadPool(int size)`
- `static ExecutorService newCachedThreadPool()`

Each of these factory methods creates and returns an object instance that implements the `ExecutorService` interface. `ExecutorService` extends the `Executor` interface, allowing us to invoke the `execute()` method on this object. In addition, `ExecutorService` provides methods for managing termination of the thread pool.

The example shown in Figure 3.5.1 creates a cached thread pool and submits tasks to be executed by a thread in the pool using the `execute()` method. When the `shutdown()` method is invoked, the thread pool rejects additional tasks and shuts down once all existing tasks have completed execution.

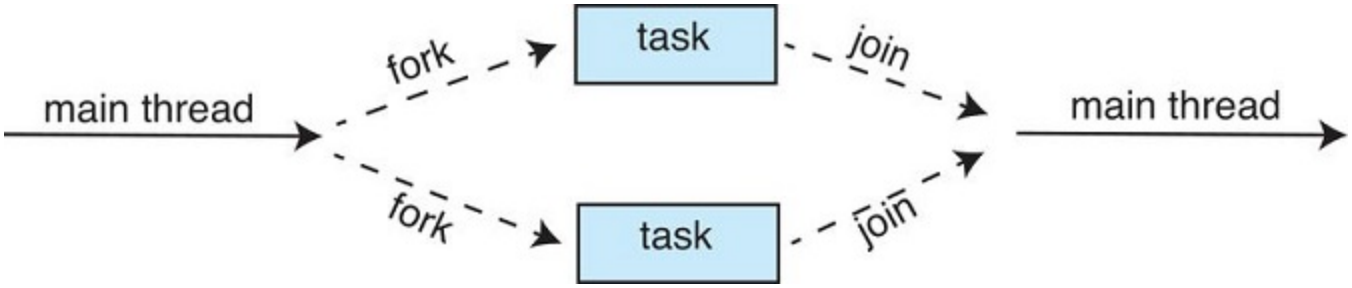
Figure 3.5.1: Creating a thread pool in Java.

```
import java.util.concurrent.*;

public class ThreadPoolExample
{
public static void main(String[] args) {
  int numTasks = Integer.parseInt(args[0].trim());

  /* Create the thread pool */
  ExecutorService pool = Executors.newCachedThreadPool();

  /* Run each task using a thread in the pool */
  for (int i = 0; i < numTasks; i++)
    pool.execute(new Task());

  /* Shut down the pool once all threads have completed */
  pool.shutdown();
}
```


Fork join

The strategy for thread creation covered in Section 3.4 is often known as the ***fork-join*** model. Recall that with this method, the main parent thread creates (***forks***) one or more child threads and then waits for the children to terminate and ***join*** with it, at which point it can retrieve and combine their results. This synchronous model is often characterized as explicit thread creation, but it is also an excellent candidate for implicit threading. In the latter situation, threads are not constructed directly during the fork stage; rather, parallel tasks are designated. This model is illustrated in Figure 3.5.2. A library manages the number of threads that are created and is also responsible for assigning tasks to threads. In some ways, this fork-join model is a synchronous version of thread pools in which a library determines the actual number of threads to create—for example, by using the heuristics described in Section Thread pools.

Figure 3.5.2: Fork-join parallelism.



Fork join in java

Java introduced a fork-join library in Version 1.7 of the API that is designed to be used with recursive divide-and-conquer algorithms such as Quicksort and Mergesort. When implementing divide-and-conquer algorithms using this library, separate tasks are forked during the divide step and assigned smaller subsets of the original problem. Algorithms must be designed so that these separate tasks can execute concurrently. At some point, the size of the problem assigned to a task is small enough that it can be solved directly and requires creating no additional tasks. The general recursive algorithm behind Java's fork-join model is shown below:

```
Task(problem)
  if problem is small enough
     solve the problem directly
  else
    subtask1 = fork(new Task(subset of problem)
    subtask2 = fork(new Task(subset of problem)

    result1 = join(subtask1)
    result2 = join(subtask2)

    return combined results
```


The animation below depicts the model graphically.

PARTICIPATION ACTIVITY 3.5.2: Fork-join in Java.

Animation content:

Step 1: A task starts. Step 2: It forks a task. Step 3: And another, while the first child forks task children of its own. Step 4: The second child spawns more tasks. Meanwhile, some children are completing their work. Step 5: More children complete, joining back to their parent. Step 6: As tasks complete, children complete and join their parent. Step 7: When all children finish their tasks, the spawning parent continue its operation.

Animation captions:

1. A task starts.
2. It forks a task.
3. And another, while the first child forks task children of its own.
4. The second child spawns more tasks. Meanwhile, some children are completing their work.
5. More children complete, joining back to their parent.
6. As tasks complete, children complete and join their parent.
7. When all children finish their tasks, the spawning parent continues its operation.

We now illustrate Java's fork-join strategy by designing a divide-and-conquer algorithm that sums all elements in an array of integers. In Version 1.7 of the API, Java introduced a new thread pool—the `ForkJoinPool`—that can be assigned tasks that inherit the abstract base class `ForkJoinTask` (which for now we will assume is the `SumTask` class). The following creates a `ForkJoinPool` object and submits the initial task via its `invoke()` method:

```
ForkJoinPool pool = new ForkJoinPool();
// array contains the integers to be summed
int[] array = new int[SIZE];

SumTask task = new SumTask(0, SIZE - 1, array);
int sum = pool.invoke(task);
```


Upon completion, the initial call to `invoke()` returns the summation of `array`.

The class `SumTask`—shown in Figure 3.5.3—implements a divide-and-conquer algorithm that sums the contents of the array using fork-join. New tasks are created using the `fork()` method, and the `compute()` method specifies the computation that is performed by each task. The method `compute()` is invoked until it can directly calculate the sum of the subset it is assigned. The call to `join()` blocks until the task completes, upon which `join()` returns the results calculated in `compute()`.

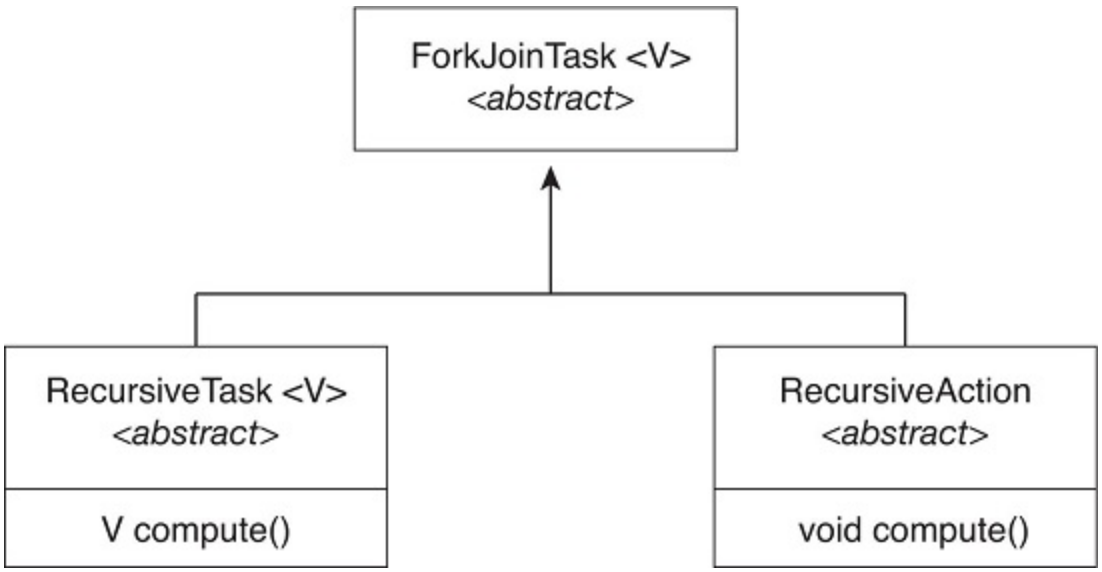
Figure 3.5.3: Fork-join calculation using the Java API.

```
import java.util.concurrent.*;

public class SumTask extends RecursiveTask<Integer>
{
  static final int THRESHOLD = 1000;

  private int begin;
  private int end;
  private int[] array;

  public SumTask(int begin, int end, int[] array) {
    this.begin = begin;
    this.end = end;
    this.array = array;
  }
  protected Integer compute() {
    if (end - begin < THRESHOLD) {
      int sum = 0;
      for (int i = begin; i <= end; i++)
        sum += array[i];

      return sum;
    }
    else {
      int mid = (begin + end) / 2;


      SumTask leftTask = new SumTask(begin, mid, array);
      SumTask rightTask = new SumTask(mid + 1, end, array);
```

```
        leftTask.fork();
        rightTask.fork();

        return rightTask.join() + leftTask.join();
      }
    }
}
```

Notice that **`SumTask`** in Figure 3.5.3 extends **`RecursiveTask`**. The Java fork-join strategy is organized around the abstract base class **`ForkJoinTask`**, and the **`RecursiveTask`** and **`RecursiveAction`** classes extend this class. The fundamental difference between these two classes is that **`RecursiveTask`** returns a result (via the return value specified in **`compute()`**), and **`RecursiveAction`** does not return a result. The relationship between the three classes is illustrated in the UML class diagram in Figure 3.5.4.

Figure 3.5.4: UML class diagram for Java's fork-join.



An important issue to consider is determining when the problem is "small enough" to be solved directly and no longer requires creating additional tasks. In **`SumTask`**, this occurs when the number of elements being summed is less than the value **`THRESHOLD`**, which in Figure 3.5.3 we have arbitrarily set to 1,000. In practice, determining when a problem can be solved directly requires careful timing trials, as the value can vary according to implementation.

What is interesting in Java's fork-join model is the management of tasks wherein the library constructs a pool of worker threads and balances the load of tasks among the available workers. In some situations, there are thousands of tasks, yet only a handful of threads performing the work (for example, a separate thread for each CPU). Additionally, each thread in a **`ForkJoinPool`** maintains a queue of tasks that it has forked, and if a thread's queue is empty, it can steal a task from another thread's queue using a ***work stealing*** algorithm, thus balancing the workload of tasks among all threads.

OpenMP

OpenMP is a set of compiler directives as well as an API for programs written in C, C++, or FORTRAN that provides support for parallel programming in shared-memory environments. OpenMP identifies ***parallel regions*** as blocks of code that may run in parallel. Application developers insert compiler directives into their code at parallel regions, and these directives instruct the OpenMP run-time library to execute the region in parallel. The following C program illustrates a compiler directive above the parallel region containing the `printf()` statement:

```
#include <omp.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
   /* sequential code */
   #pragma omp parallel
   {
     printf("I am a parallel region.")
   }

   /* sequential code */
   return 0
}
```

When OpenMP encounters the directive

```
#pragma omp parallel
```

it creates as many threads as there are processing cores in the system. Thus, for a dual-core system, two threads are created; for a quad-core system, four are created; and so forth. All the threads then simultaneously execute the parallel region. As each thread exits the parallel region, it is terminated.

OpenMP provides several additional directives for running code regions in parallel, including parallelizing loops. For example, assume we have two arrays, `a` and `b`, of size `N`. We wish to sum their contents and place the results in array `c`. We can have this task run in parallel by using the following code segment, which contains the compiler directive for parallelizing `for` loops:

```
#pragma omp parallel for
for (i = 0; i < N; i++) {
  c[i] = a[i] + b[i];
}
```

OpenMP divides the work contained in the `for` loop among the threads it has created in response to the directive

```
#pragma omp parallel for
```

In addition to providing directives for parallelization, OpenMP allows developers to choose among several levels of parallelism. For example, they can set the number of threads manually. It also allows developers

to identify whether data are shared between threads or are private to a thread. OpenMP is available on several open-source and commercial compilers for Linux, Windows, and macOS systems. We encourage readers interested in learning more about OpenMP to consult the bibliography at the end of the chapter.

Grand central dispatch

Grand Central Dispatch (GCD) is a technology developed by Apple for its macOS and iOS operating systems. It is a combination of a run-time library, an API, and language extensions that allow developers to identify sections of code (***tasks***) to run in parallel. Like OpenMP, GCD manages most of the details of threading.

GCD schedules tasks for run-time execution by placing them on a ***dispatch queue***. When it removes a task from a queue, it assigns the task to an available thread from a pool of threads that it manages. GCD identifies two types of dispatch queues: ***serial*** and ***concurrent***.

Tasks placed on a serial queue are removed in FIFO order. Once a task has been removed from the queue, it must complete execution before another task is removed. Each process has its own serial queue (known as its ***main queue***), and developers can create additional serial queues that are local to a particular process. (This is why serial queues are also known as ***private dispatch queues***.) Serial queues are useful for ensuring the sequential execution of several tasks.

Tasks placed on a concurrent queue are also removed in FIFO order, but several tasks may be removed at a time, thus allowing multiple tasks to execute in parallel. There are several system-wide concurrent queues (also known as ***global dispatch queues***), which are divided into four primary quality-of-service classes:

- `QOS_CLASS_USER_INTERACTIVE`—The ***user-interactive*** class represents tasks that interact with the user, such as the user interface and event handling, to ensure a responsive user interface. Completing a task belonging to this class should require only a small amount of work.
- `QOS_CLASS_USER_INITIATED`—The ***user-initiated*** class is similar to the user-interactive class in that tasks are associated with a responsive user interface; however, user-initiated tasks may require longer processing times. Opening a file or a URL is a user-initiated task, for example. Tasks belonging to this class must be completed for the user to continue interacting with the system, but they do not need to be serviced as quickly as tasks in the user-interactive queue.
- `QOS_CLASS_UTILITY` — The ***utility*** class represents tasks that require a longer time to complete but do not demand immediate results. This class includes work such as importing data.
- `QOS_CLASS_BACKGROUND` — Tasks belonging to the ***background*** class are not visible to the user and are not time sensitive. Examples include indexing a mailbox system and performing backups.

Tasks submitted to dispatch queues may be expressed in one of two different ways:

1. For the C, C++, and Objective-C languages, GCD identifies a language extension known as a ***block***, which is simply a self-contained unit of work. A block is specified by a caret ^ inserted in front of a pair of braces { }. Code within the braces identifies the unit of work to be performed. A simple example of a block is shown below:

```
^{ printf("I am a block"); }
```

2. For the Swift programming language, a task is defined using a ***closure***, which is similar to a block in that it expresses a self-contained unit of functionality. Syntactically, a Swift closure is written in the same way as a block, minus the leading caret.

   The following Swift code segment illustrates obtaining a concurrent queue for the user-initiated class and submitting a task to the queue using the `dispatch_async()` function:

```
let queue = dispatch_get_global_queue
  (QOS_CLASS_USER_INITIATED, 0)

dispatch_async(queue,{ print("I am a closure.") })
```

Internally, GCD's thread pool is composed of POSIX threads. GCD actively manages the pool, allowing the number of threads to grow and shrink according to application demand and system capacity. GCD is implemented by the `libdispatch` library, which Apple has released under the Apache Commons license. It has since been ported to the FreeBSD operating system.

Intel thread building blocks

Intel threading building blocks (TBB) is a template library that supports designing parallel applications in C++. As this is a library, it requires no special compiler or language support. Developers specify tasks that can run in parallel, and the TBB task scheduler maps these tasks onto underlying threads. Furthermore, the task scheduler provides load balancing and is cache aware, meaning that it will give precedence to tasks that likely have their data stored in cache memory and thus will execute more quickly. TBB provides a rich set of features, including templates for parallel loop structures, atomic operations, and mutual exclusion locking. In addition, it provides concurrent data structures, including a hash map, queue, and vector, which can serve as equivalent thread-safe versions of the C++ standard template library data structures.

Let's use parallel for loops as an example. Initially, assume there is a function named `apply(float value)` that performs an operation on the parameter `value`. If we had an array `v` of size `n` containing `float` values, we could use the following serial for loop to pass each value in `v` to the `apply()` function:

```
for (int i = 0; i < n; i++) {
  apply(v[i]);
}
```


A developer could manually apply data parallelism (Section Types of parallelism) on a multicore system by assigning different regions of the array `v` to each processing core; however, this ties the technique for achieving parallelism closely to the physical hardware, and the algorithm would have to be modified and recompiled for the number of processing cores on each specific architecture.

Alternatively, a developer could use TBB, which provides a `parallel_for` template that expects two values:

```
parallel_for (range   body)
```

where ***range*** refers to the range of elements that will be iterated (known as the ***iteration space***) and ***body*** specifies an operation that will be performed on a subrange of elements.

We can now rewrite the above serial for loop using the TBB `parallel_for` template as follows:

```
parallel_for (size_t(0), n, [=](size_t i) {apply(v[i]);});
```


The first two parameters specify that the iteration space is from 0 to `n` − 1 (which corresponds to the number of elements in the array `v`). The second parameter is a C++ lambda function that requires a bit of explanation. The expression `[=](size_t i)` is the parameter `i`, which assumes each of the values over the iteration space (in this case from 0 to `n` − 1). Each value of `i` is used to identify which array element in `v` is to be passed as a parameter to the `apply(v[i])` function.

The TBB library will divide the loop iterations into separate "chunks" and create a number of tasks that operate on those chunks. (The `parallel_for` function allows developers to manually specify the size of the chunks if they wish to.) TBB will also create a number of threads and assign tasks to available threads. This is quite similar to the fork-join library in Java. The advantage of this approach is that it requires only that developers identify what operations can run in parallel (by specifying a `parallel_for` loop), and the library manages the details involved in dividing the work into separate tasks that run in parallel. Intel TBB has both commercial and open-source versions that run on Windows, Linux, and macOS. Refer to the bibliography for further details on how to develop parallel applications using TBB.

PARTICIPATION ACTIVITY 3.5.3: Section review questions.

1) When OpenMP encounters the #pragma omp parallel directive, it
   - ○ constructs a parallel region.
   - ○ creates a new thread.
   - ○ creates as many threads as there are processing cores.

2) Grand Central Dispatch handles blocks by
   - ○ placing them on a dispatch queue.
   - ○ creating a new thread.
   - ○ placing them on a dispatch stack.

Section glossary

implicit threading: A programming model that transfers the creation and management of threading from application developers to compilers and run-time libraries.

thread pool: A number of threads created at process startup and placed in a pool, where they sit and wait for work.

fork-join: A strategy for thread creation in which the main parent thread creates (forks) one or more child threads and then waits for the children to terminate and join with it.

parallel regions: Blocks of code that may run in parallel.

dispatch queue: An Apple OS feature for parallelizing code; blocks are placed in the queue by Grand Central Dispatcher (GCD) and removed to be run by an available thread.

main queue: Apple OS per-process block queue.

user-interactive: In the Grand Central Dispatch Apple OS scheduler, the scheduling class representing tasks that interact with the user.

user-initiated: In the Grand Central Dispatch Apple OS scheduler, the scheduling class representing tasks that interact with the user but need longer processing times than user-interactive tasks.

utility: In the Grand Central Dispatch Apple OS scheduler, the scheduling class representing tasks that require a longer time to complete but do not demand immediate results.

background: Describes a process or thread that is not currently interactive (has no interactive input directed to it), such as one not currently being used by a user. In the Grand Central Dispatch Apple OS scheduler, the scheduling class representing tasks that are not time sensitive and are not visible to the user.

block: A self-contained unit of work. The smallest physical storage device storage unit, typically 512B or 4KB. In the Grand Central Dispatch Apple OS scheduler, a language extension that allows designation of a section of code that can be submitted to dispatch queues.

iteration space: In Intel threading building blocks, the range of elements that will be iterated.

3.6 Threading issues

In this section, we discuss some of the issues to consider in designing multithreaded programs.

The `fork()` and `exec()` system calls

In the previous chapter Processes, we described how the `fork()` system call is used to create a separate, duplicate process. The semantics of the `fork()` and `exec()` system calls change in a multithreaded program.

If one thread in a program calls `fork()`, does the new process duplicate all threads, or is the new process single-threaded? Some UNIX systems have chosen to have two versions of `fork()`, one that duplicates all threads and another that duplicates only the thread that invoked the `fork()` system call.

The `exec()` system call typically works in the same way as described in the chapter Processes. That is, if a thread invokes the `exec()` system call, the program specified in the parameter to `exec()` will replace the entire process—including all threads.

Which of the two versions of `fork()` to use depends on the application. If `exec()` is called immediately after forking, then duplicating all threads is unnecessary, as the program specified in the parameters to `exec()` will replace the process. In this instance, duplicating only the calling thread is appropriate. If, however, the separate process does not call `exec()` after forking, the separate process should duplicate all threads.

Signal handling

A ***signal*** is used in UNIX systems to notify a process that a particular event has occurred. A signal may be received either synchronously or asynchronously, depending on the source of and the reason for the event being signaled. All signals, whether synchronous or asynchronous, follow the same pattern:

1. A signal is generated by the occurrence of a particular event.
2. The signal is delivered to a process.
3. Once delivered, the signal must be handled.

Examples of synchronous signals include illegal memory access and division by 0. If a running program performs either of these actions, a signal is generated. Synchronous signals are delivered to the same process that performed the operation that caused the signal (that is the reason they are considered synchronous).

When a signal is generated by an event external to a running process, that process receives the signal asynchronously. Examples of such signals include terminating a process with specific keystrokes (such as `<control><C>`) and having a timer expire. Typically, an asynchronous signal is sent to another process.

A signal may be ***handled*** by one of two possible handlers:

1. A default signal handler
2. A user-defined signal handler

Every signal has a ***default signal handler*** that the kernel runs when handling that signal. This default action can be overridden by a ***user-defined signal handler*** that is called to handle the signal. Signals are handled in different ways. Some signals may be ignored, while others (for example, an illegal memory access) are handled by terminating the program.

Handling signals in single-threaded programs is straightforward: signals are always delivered to a process. However, delivering signals is more complicated in multithreaded programs, where a process may have several threads. Where, then, should a signal be delivered?

In general, the following options exist:

1. Deliver the signal to the thread to which the signal applies.
2. Deliver the signal to every thread in the process.
3. Deliver the signal to certain threads in the process.
4. Assign a specific thread to receive all signals for the process.

The method for delivering a signal depends on the type of signal generated. For example, synchronous signals need to be delivered to the thread causing the signal and not to other threads in the process. However, the situation with asynchronous signals is not as clear. Some asynchronous signals—such as a signal that terminates a process (**<control><C>**, for example)—should be sent to all threads.

The standard UNIX function for delivering a signal is

```
kill(pid_t pid, int signal)
```

This function specifies the process (**pid**) to which a particular signal (**signal**) is to be delivered. Most multithreaded versions of UNIX allow a thread to specify which signals it will accept and which it will block. Therefore, in some cases, an asynchronous signal may be delivered only to those threads that are not blocking it. However, because signals need to be handled only once, a signal is typically delivered only to the first thread found that is not blocking it. POSIX Pthreads provides the following function, which allows a signal to be delivered to a specified thread (**tid**):

```
pthread_kill(pthread_t tid, int signal)
```

Although Windows does not explicitly provide support for signals, it allows us to emulate them using ***asynchronous procedure calls*** (*APCs*). The APC facility enables a user thread to specify a function that is to be called when the user thread receives notification of a particular event. As indicated by its name, an APC is roughly equivalent to an asynchronous signal in UNIX. However, whereas UNIX must contend with how to deal with signals in a multithreaded environment, the APC facility is more straightforward, since an APC is delivered to a particular thread rather than a process.

PARTICIPATION ACTIVITY 3.6.1: Mid-section Review Question.

Terminating a running application using a keystroke sequence causes a(n) _____ exception.

- synchronous
- asynchronous

Thread cancellation

Thread cancellation involves terminating a thread before it has completed. For example, if multiple threads are concurrently searching through a database and one thread returns the result, the remaining threads might be canceled. Another situation might occur when a user presses a button on a web browser that stops a web page from loading any further. Often, a web page loads using several threads—each image is loaded in a separate thread. When a user presses the `stop` button on the browser, all threads loading the page are canceled.

A thread that is to be canceled is often referred to as the ***target thread***. Cancellation of a target thread may occur in two different scenarios:

1. **Asynchronous cancellation.** One thread immediately terminates the target thread.
2. **Deferred cancellation.** The target thread periodically checks whether it should terminate, allowing it an opportunity to terminate itself in an orderly fashion.

The difficulty with cancellation occurs in situations where resources have been allocated to a canceled thread or where a thread is canceled while in the midst of updating data it is sharing with other threads. This becomes especially troublesome with asynchronous cancellation. Often, the operating system will reclaim system resources from a canceled thread but will not reclaim all resources. Therefore, canceling a thread asynchronously may not free a necessary system-wide resource or leave data in an inconsistent state.

With deferred cancellation, in contrast, one thread indicates that a target thread is to be canceled, but cancellation occurs only after the target thread has checked a flag to determine whether or not it should be canceled. The thread can perform this check at a point at which it can be canceled safely.

In Pthreads, thread cancellation is initiated using the `pthread_cancel()` function. The identifier of the target thread is passed as a parameter to the function. The following code illustrates creating—and then canceling—a thread:

```
pthread_t tid;

/* create the thread */
pthread_create(&tid, 0, worker, NULL);

 . . .

/* cancel the thread */
pthread_cancel(tid);

/* wait for the thread to terminate */
pthread_join(tid,NULL);
```


Invoking `pthread_cancel()` indicates only a request to cancel the target thread, however; actual cancellation depends on how the target thread is set up to handle the request. When the target thread is finally canceled, the call to `pthread_join()` in the canceling thread returns. Pthreads supports three

cancellation modes. Each mode is defined as a state and a type, as illustrated in the table below. A thread may set its cancellation state and type using an API.

Mode	State	Type
Off	Disabled	–
Deferred	Enabled	Deferred
Asynchronous	Enabled	Asynchronous

As the table illustrates, Pthreads allows threads to disable or enable cancellation. Obviously, a thread cannot be canceled if cancellation is disabled. However, cancellation requests remain pending, so the thread can later enable cancellation and respond to the request.

The default cancellation type is deferred cancellation. However, cancellation occurs only when a thread reaches a ***cancellation point***. Most of the blocking system calls in the POSIX and standard C library are defined as cancellation points, and these are listed when invoking the command `man pthreads` on a Linux system. For example, the `read()` system call is a cancellation point that allows cancelling a thread that is blocked while awaiting input from `read()`.

One technique for establishing a cancellation point is to invoke the `pthread_testcancel()` function. If a cancellation request is found to be pending, the call to `pthread_testcancel()` will not return, and the thread will terminate; otherwise, the call to the function will return, and the thread will continue to run. Additionally, Pthreads allows a function known as a ***cleanup handler*** to be invoked if a thread is canceled. This function allows any resources a thread may have acquired to be released before the thread is terminated.

The following code illustrates how a thread may respond to a cancellation request using deferred cancellation:

```
while (1) {
   /* do some work for awhile */

    . . .

/* check if there is a cancellation request */
pthread_testcancel();
}
```

Because of the issues described earlier, asynchronous cancellation is not recommended in Pthreads documentation. Thus, we do not cover it here. An interesting note is that on Linux systems, thread cancellation using the Pthreads API is handled through signals (Section Signal handling).

Thread cancellation in Java uses a policy similar to deferred cancellation in Pthreads. To cancel a Java thread, you invoke the `interrupt()` method, which sets the interruption status of the target thread to true:

```
Thread worker;
```

```
 . . .

/* set the interruption status of the thread */
worker.interrupt()
```

A thread can check its interruption status by invoking the `isInterrupted()` method, which returns a boolean value of a thread's interruption status:

```
while (!Thread.currentThread().isInterrupted()) {
   . . .
}
```

PARTICIPATION ACTIVITY 3.6.2: Mid-section Review Question.

Cancellation points are associated with ____ cancellation.

- synchronous
- asynchronous
- deferred

Thread-local storage

Threads belonging to a process share the data of the process. Indeed, this data sharing provides one of the benefits of multithreaded programming. However, in some circumstances, each thread might need its own copy of certain data. We will call such data ***thread-local storage*** (or *TLS*). For example, in a transaction-processing system, we might service each transaction in a separate thread. Furthermore, each transaction might be assigned a unique identifier. To associate each thread with its unique transaction identifier, we could use thread-local storage.

It is easy to confuse TLS with local variables. However, local variables are visible only during a single function invocation, whereas TLS data are visible across function invocations. Additionally, when the developer has no control over the thread creation process—for example, when using an implicit technique such as a thread pool—then an alternative approach is necessary.

In some ways, TLS is similar to `static` data; the difference is that TLS data are unique to each thread. (In fact, TLS is usually declared as `static`.) Most thread libraries and compilers provide support for TLS. For example, Java provides a `ThreadLocal<T>` class with `set()` and `get()` methods for `ThreadLocal<T>` objects. Pthreads includes the type `pthread_key_t`, which provides a key that is specific to each thread. This key can then be used to access TLS data. Microsoft's C# language simply requires adding the storage attribute `[ThreadStatic]` to declare thread-local data. The `gcc` compiler provides the storage class keyword `_thread` for declaring TLS data. For example, if we wished to assign a unique identifier for each thread, we would declare it as follows:

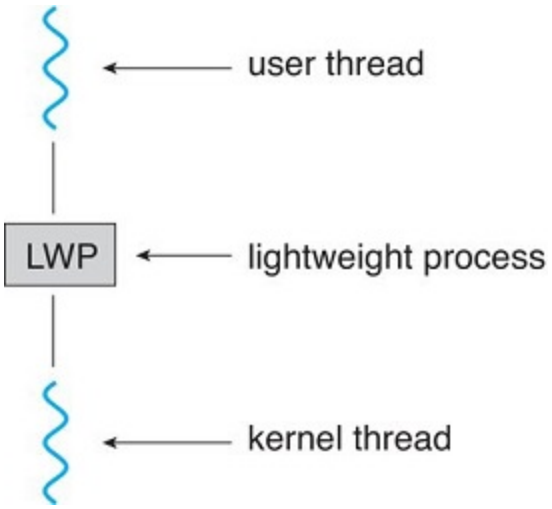
```
static _thread int threadID;
```

Scheduler activations

A final issue to be considered with multithreaded programs concerns communication between the kernel and the thread library, which may be required by the many-to-many and two-level models discussed in Section Many-to-many model. Such coordination allows the number of kernel threads to be dynamically adjusted to help ensure the best performance.

Many systems implementing either the many-to-many or the two-level model place an intermediate data structure between the user and kernel threads. This data structure—typically known as a ***lightweight process***, or *LWP*—is shown in Figure 3.6.1. To the user-thread library, the LWP appears to be a virtual processor on which the application can schedule a user thread to run. Each LWP is attached to a kernel thread, and it is kernel threads that the operating system schedules to run on physical processors. If a kernel thread blocks (such as while waiting for an I/O operation to complete), the LWP blocks as well. Up the chain, the user-level thread attached to the LWP also blocks.

Figure 3.6.1: Lightweight process (LWP).



An application may require any number of LWPs to run efficiently. Consider a CPU-bound application running on a single processor. In this scenario, only one thread can run at a time, so one LWP is sufficient. An application that is I/O-intensive may require multiple LWPs to execute, however. Typically, an LWP is required for each concurrent blocking system call. Suppose, for example, that five different file-read requests occur simultaneously. Five LWPs are needed, because all could be waiting for I/O completion in the kernel. If a process has only four LWPs, then the fifth request must wait for one of the LWPs to return from the kernel.

One scheme for communication between the user-thread library and the kernel is known as ***scheduler activation***. It works as follows: The kernel provides an application with a set of virtual processors (LWPs), and the application can schedule user threads onto an available virtual processor. Furthermore, the kernel

must inform an application about certain events. This procedure is known as an ***upcall***. Upcalls are handled by the thread library with an ***upcall handler***, and upcall handlers must run on a virtual processor.

One event that triggers an upcall occurs when an application thread is about to block. In this scenario, the kernel makes an upcall to the application informing it that a thread is about to block and identifying the specific thread. The kernel then allocates a new virtual processor to the application. The application runs an upcall handler on this new virtual processor, which saves the state of the blocking thread and relinquishes the virtual processor on which the blocking thread is running. The upcall handler then schedules another thread that is eligible to run on the new virtual processor. When the event that the blocking thread was waiting for occurs, the kernel makes another upcall to the thread library informing it that the previously blocked thread is now eligible to run. The upcall handler for this event also requires a virtual processor, and the kernel may allocate a new virtual processor or preempt one of the user threads and run the upcall handler on its virtual processor. After marking the unblocked thread as eligible to run, the application schedules an eligible thread to run on an available virtual processor.

PARTICIPATION ACTIVITY 3.6.3: Section review questions.

1) Thread-local storage is ____.
   - another term for local variables
   - data that has been modified by a thread, but not yet updated to all other threads belonging to the same process
   - data that is unique to each thread

2) _____ is the preferred form of cancellation.
   - Deferred cancellation
   - Asynchronous cancellation

3) Deferred cancellation is guaranteed to ultimately terminate the target thread.
   - True
   - False

Section glossary

signal: In UNIX and other operating systems, a means used to notify a process that an event has occurred.

default signal handler: The signal handler that receives signals unless a user-defined signal handler is provided by a process.

user-defined signal handler: The signal handler created by a process to provide non-default signal handling.

asynchronous procedure call (APC): A facility that enables a user thread to specify a function that is to be called when the user thread receives notification of a particular event.

thread cancellation: Termination of a thread before it has completed.

cancellation point: With deferred thread cancellation, a point in the code at which it is safe to terminate the thread.

clean-up handler: A function that allows any resources a thread has acquired to be released before the thread is terminated.

thread-local storage (TLS): Data available only to a given thread.

lightweight process (LWP): A virtual processor-like data structure allowing a user thread to map to a kernel thread.

scheduler activation: A threading method in which the kernel provides an application with a set of LWPs, and the application can schedule user threads onto an available virtual processor and receive upcalls from the kernel to be informed of certain events.

upcall: A threading method in which the kernel sends a signal to a process thread to communicate an event.

upcall handler: A function in a process that handles upcalls.

3.7 Threads

The thread concept

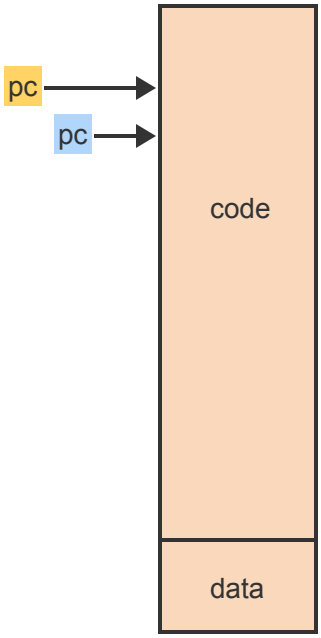
An application implemented as a single process follows a single path of execution through the program. Whenever the execution blocks, waiting for some resource to become available, the entire process blocks. Many applications have parts that could run concurrently if only each could block independently. Similarly, independent parts could run in parallel on multiple CPUs.

Creating a separate process for each of potentially many short-lived activities is too inefficient. A ***thread*** is an instance of executing a portion of a program within a process without incurring the overhead of creating and managing separate PCBs.

Example 3.7.1: Typical uses of threads.

- A browser can draw an image on the screen while at the same time waiting for data from the Internet. One thread can be busy drawing the image while another can keep blocking while retrieving the data.
- A word processor can run a spell checker at the same time as the user is typing. Waiting for each typed character and displaying the data on the screen can be done concurrently by two separate threads.
- An Internet server, such as an email or web page server, is receiving many requests from different users at the same time. When each request is implemented as a new thread, many requests can proceed concurrently without holding up the progress of others.

PARTICIPATION ACTIVITY 3.7.1: Multi-threaded execution of the same code.



Animation content:

Static figure: A simplified representation of a computer process's memory space, consisting of two separate sections, code and data.

Step 1: A single-threaded process has a single program counter (pc).
A small box labeled pc appears. An arrow points from the pc to the top of the code section.

Step 2: The pc moves through the program as execution progresses.
The pc arrow moves further down and then back up again along the code section.

Step 3: A process with multiple threads has a separate pc for each thread.
Another pc box appears. The arrow points at a different point within the code section.

Step 4: The different pc's advance through the code independently from one another as each thread executes a different sequence of instructions within the same code.
The two pc arrows alternate in moving up and down to various points within the code section.

Step 5: When multiple CPUs are available, the threads can proceed in parallel.
The two pc arrows move concurrently up and down to various points within the code section.

Animation captions:

1. A single-threaded process has a single program counter (pc).
2. The pc moves through the program as execution progresses.
3. A process with multiple threads has a separate pc for each thread.
4. The different pc's advance through the code independently from one another as each thread executes a different sequence of instructions within the same code.
5. When multiple CPUs are available, the threads can proceed in parallel.

PARTICIPATION ACTIVITY 3.7.2: Threads within a process.

Multiple threads within a process _____.

- will always execute the same sequence of instructions
- must execute within disjoint parts of the shared program
- may execute different sequences of instructions within any part of the shared program

PARTICIPATION ACTIVITY 3.7.3: Threads vs processes.

Using n threads within a single process is more efficient than using n separate processes because _____.

1) the threads share the same code and data

- True
- False

2) only the threads can take advantage of multiple CPUs

- True
- False

3) only threads can block independently from one another

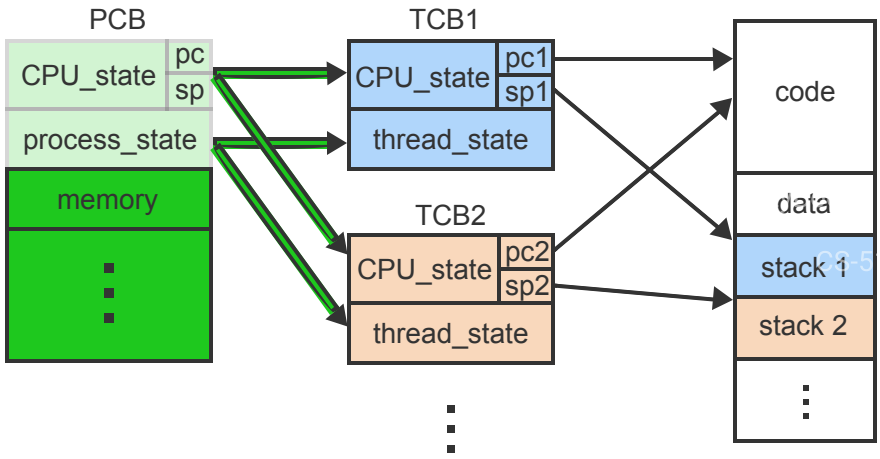
- True
- False

The thread control block

Since each thread within a process is an independent execution activity, the runtime information held in the PCB and the execution stack must be replicated for each thread. A ***thread control block (TCB)*** is a data structure that holds a separate copy of the dynamically changing information necessary for a thread to execute independently.

The replication of only the bare minimum of information in each TCB, while sharing the same code, global data, resources, and open files, is what makes threads much more efficient to manage than processes.

PARTICIPATION ACTIVITY 3.7.4: A multi-threaded process.



Animation content:

Static figure: A conceptual model of the process and thread management in an operating system depicted in 3 columns. Column 1 contains a single rectangle labeled PCB, which consist of a CPU_state with pc and sp, a process_state, and memory. Column 2 contains 2 blocks labeled TCB1 and TCB2. Each consists of a CPU_state and a thread_state. The CPU_state of TCB1 contains pc1 and sp1. The CPU_state of TCB2 contains pc2 and sp2. Column 3 contains a block divided into code, data, stack 1, and stack 2.
Arrows point from CPU_state in PCB to CPU_state in TCB1 and TCB2. Arrows point also from Process_state in PCB to thread state in TCB1 and TCB2. Pc1 and pc2 both point to the code section in Column 3. Sp1 and sp2 point to stack1 and stack 2, respectively.

Step 1: A single-threaded process has one program counter (pc) and one stack pointer (sp) as part of the CPU_state. The process_state may be running, ready, blocked. PCB in column 1 appears.

Step 2: The CPU_state is replicated in each TCB, giving each TCB a private pc and sp.
The CPU_states of both TCB1 and TCB2 appear and are pointed at from the CPU_state of the PCB. The CPU_states contain pc1/sp1 and pc2/sp2, respectively.

Step 3: The process_state is replicated as a thread_state in each TCB. Thus each thread can be running/ready/blocked independently. Other fields (memory, ...) remain in PCB.
The thread_states of both TCB1 and TCB2 appear and are pointed at from the process_state of the PCB.

Step 4: Each thread accesses the shared code using the private pc. Each thread also has a private stack, accessed using the private sp. Column 3 appears. Pc1 and pc2 both point to the code section in Column 3. Sp1 and sp2 point to stack1 and stack 2, respectively.

Animation captions:

1. A single-threaded process has one program counter (pc) and one stack pointer (sp) as part of the CPU_state. The process_state may be running, ready, blocked.
2. The CPU_state is replicated in each TCB, giving each TCB a private pc and sp.
3. The process_state is replicated as a thread_state in each TCB. Thus each thread can be running/ready/blocked independently. Other fields (memory, ...) remain in PCB.
4. Each thread accesses the shared code using the private pc. Each thread also has a private stack, accessed using the private sp.

PARTICIPATION ACTIVITY 3.7.5: TCBs vs PCB.

1) Threads are faster to create and destroy than processes.

- True
- False

2) In a multi-threaded process the PCB is replaced by multiple TCBs.

- True
- False

3) Memory and other fields are not replicated in the TCBs because the data is not needed by threads.

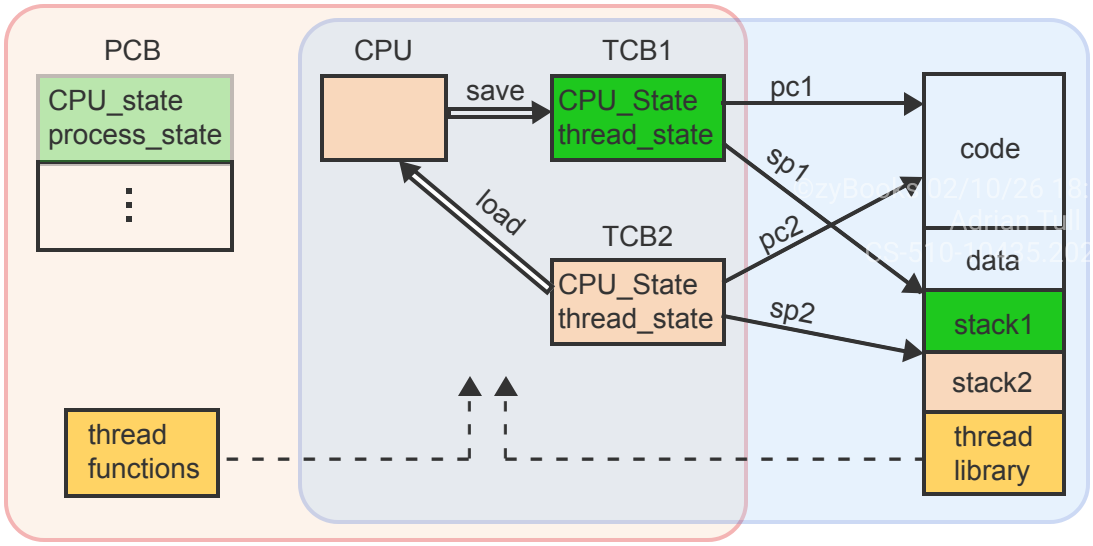
- True
- False

User-level vs kernel-level threads

Threads can be implemented completely within a user application. A thread library provides functions to create, destroy, schedule, and coordinate threads. The library maintains all TCBs within the user process. Consequently, the OS kernel is not aware of the threads and treats the process as a single-threaded execution.

The alternative is to implement threads within the OS kernel. The TCBs are not directly accessible to the application, which uses kernel calls to create, destroy, and otherwise manipulate the threads.

PARTICIPATION ACTIVITY 3.7.6: User-level vs kernel-level threads.



Kernel-level thread management

User-level thread management

Animation content:

Static figure: The components involved in thread management in an Operating Systems context. The main elements shown are: in column 1 a PCB, in column 2 a CPU, in column 3 TCB1 and TCB2, in column 4 a block divided into code, data, stack 1, stack 2, and thread library.
Each of these blocks contains various elements that represent the state of the process or thread. The PCB contains labeled blocks for CPU_state and process_state, and the TCBs contain CPU_State and thread_state. There are arrows depicting actions such as save and load between the CPU and the TCBs. Arrows labeled pc1 and pc2 point from the TCBs to the code section on column 4, arrows labeled sp1 and sp2 point from the TCBs to stack1 and stack 2 respectively.
Columns 1 through 3 are encircled by a box labeled Kernel-level thread management.
Columns 2 through 4 are encircled by a box labeled User-level thread management.

Step 1: When the process starts running, the saved CPU_state is copied into the CPU. PCB and CPU (columns 1 and 2) appear. An arrow from the CPU_state in the PCB to the CPU indicates the act of copying.

Step 2: The CPU continues executing the code using the current program counter (pc) and stack pointer (sp). The PCB gets out of date as the process executes. Column 4 appears. Arrows labeled pc and sp point from the CPU to code and stack1 in column 4. The CPU_state in PCB is deemphasized.

Step 3: Using a thread library, the application can create multiple threads, each using a different pc and different sp. All threads share the code and global data.
TCB1 and TCB2 appear. Arrows pc1 and pc2 point to code. Arrows sp1 and sp2 point to stack1 and stack2 respectively.

Step 4: The thread library switches between the threads by saving and copying the CPU_states between the TCBs and the CPU. An arrow labeled save points from CPU to TCB1 and an arrow labeled load points from TCB2 to CPU.

Step 5: With user-level threads, all thread management occurs within the user process. The kernel is not aware of the multiple threads. Columns 2 through 4 are encircled by a box labeled User-level thread management.

Step 6: With kernel-level threads, the thread management is handled by the kernel using internal thread functions. The application uses kernel calls to request any action. A box labeled Kernal functions appears in column 1. Columns 1 through 3 are encircled by a box labeled Kernel-level thread management.

Animation captions:

1. When the process starts running, the saved CPU_state is copied into the CPU.
2. The CPU continues executing the code using the current program counter (pc) and stack pointer (sp). The PCB gets out of date as the process executes.
3. Using a thread library, the application can create multiple threads, each using a different pc and different sp. All threads share the code and global data.
4. The thread library switches between the threads by saving and copying the CPU_states between the TCBs and the CPU.
5. With user-level threads, all thread management occurs within the user process. The kernel is not aware of the multiple threads.
6. With kernel-level threads, the thread management is handled by the kernel using internal thread functions. The application uses kernel calls to request any action.

PARTICIPATION ACTIVITY 3.7.7: Who performs a context switch.

With a multi-threaded process, a context switch between threads is performed by the OS kernel.

- Only with user-level threads.
- Only with kernel-level threads.
- Always.
- Never.

PARTICIPATION ACTIVITY 3.7.8: Process state vs thread states.

With a multi-threaded process, each TCB maintains a separate copy of the thread state (running, ready, or blocked) but the PCB must also have a copy of the process state (running, ready, or blocked).

- Only with kernel-level threads.
- Only with user-level threads.
- Always.
- Never.

Combining user-level and kernel-level threads

Advantages of user-level threads (ULTs) over kernel-level threads (KLTs):

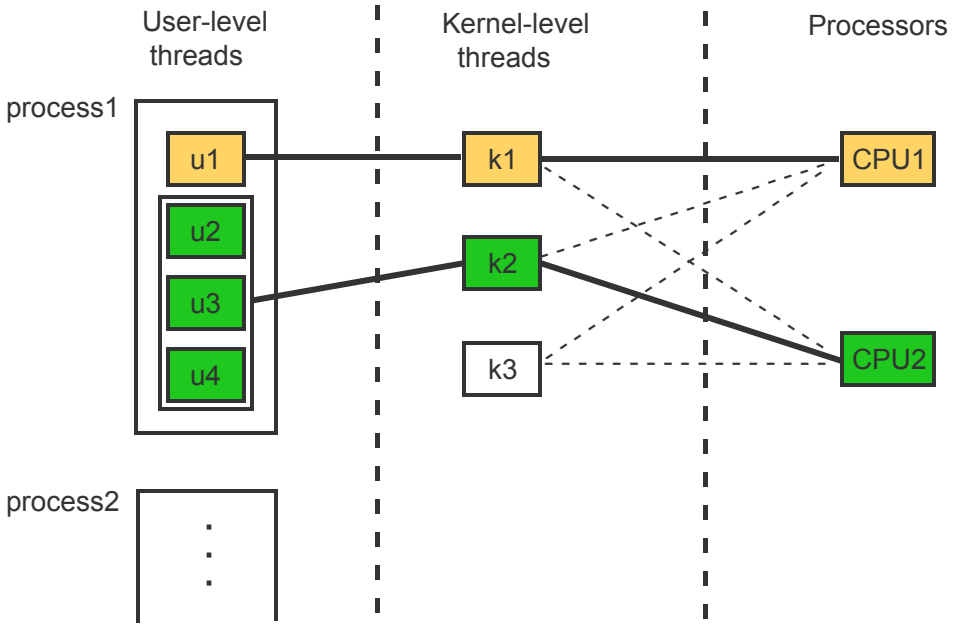
- Because ULTs do not require any cooperation from the kernel, ULTs are much faster to manage (create, destroy, and schedule) and thus many more can be created than KLTs.
- Applications using ULTs are portable between different OSs without modifications.

Main disadvantages of ULTs over KLTs:

- ULTs are not visible to the kernel. When one ULT blocks, the entire process blocks, which decreases concurrency and thus the performance and responsiveness of the application.
- ULTs cannot take advantage of multiple CPUs because the process is perceived by the kernel as a single thread of execution.

Many modern OSs take a combined approach. The kernel supports a small number of KLTs. Each application can implement any number of ULTs, which are then mapped on the KLTs based on the ULTs' needs and the available resources.

PARTICIPATION ACTIVITY 3.7.9: Mapping user-level threads on kernel-level threads.



Animation content:

Static figure: The relationship between user-level threads (ULTs), kernel-level threads (KLTs), and processors, each depicted in a separate column. The ULTs column contains 2 blocks labeled process 1 and process 2. Process 1contains four boxes representing threads (u1, u2, u3, u4). The KLTs column contains three blocks representing kernel-level threads k1, k2, and k3. The Processors column shows two blocks representing CPU1 and CPU2. Lines connect some KLTs to both ULTs and processors.

Step 1: The kernel maintains a small number of KLTs. The kernel also schedules the KLTs on the available CPUs. The KLTs and Processors columns appear.
Dashed lines connect each of the three KLTs (k1, k2, k3) to both CPU1 and CPU2.

Step 2: Each process can create any number of ULTs using a thread library. ULTs Column appears.

Step 3: The programmer decides which ULTs or groups of ULTs are mapped on separate KLTs.
ULT u1 is connected to KLT k1. ULTs u2, u3, and u4 are grouped together and connected to KLT k2.

Step 4: When u1 blocks, k1 blocks as well. But as long as at least one of u2, u3, or u4 is running, the process continues running in k2.
K1 is marked as blocked while k2 is marked as running.

Step 5: When u1 runs again and one of u2, u3, or u4 continues running then the process may continue in parallel on both CPUs. K1 is marked as running again and is connected to CPU1. K2 continues running as is connected to CPU2.

Animation captions:

1. The kernel maintains a small number of KLTs. The kernel also schedules the KLTs on the available CPUs.
2. Each process can create any number of ULTs using a thread library.
3. The programmer decides which ULTs or groups of ULTs are mapped on separate KLTs.
4. When u1 blocks, k1 blocks as well. But as long as at least one of u2, u3, or u4 is running, the process continues running in k2.
5. When u1 runs again and one of u2, u3, or u4 continues running then the process may continue in parallel on both CPUs.

PARTICIPATION ACTIVITY 3.7.10: Mapping ULTs on KLTs.

A process consisting of n ULTs is to be mapped onto a set of m KLTs. Which combination would not fully utilize available resources?

- n = m > 0
- n > m = 1
- m > n > 1
- n > m > 1

PARTICIPATION ACTIVITY 3.7.11: Kernel calls for threads.

Kernel calls are _____ to manage threads.

- needed only with KLTs
- needed only with ULTs
- always needed
- never needed

EXERCISE 3.7.1: Single-threaded vs multi-threaded computation.

A server accepts and processes requests from clients. The server keeps the results of the most recent requests in memory as a cache.

- A new request arrives on average every 25 ms.
- The processing of each request takes 15 ms.
- If the requested result is not in the memory cache, additional 75 ms are needed to access the disk.
- On average, 80% of all requests can be serviced without disk access.
- The creation of a new thread takes 10 ms.

(a) Should the server be implemented as a single-threaded or a multi-threaded process? Justify your answer.

(b) What percentage of requests would have to be satisfied without disk access for the single-threaded approach to be feasible?

EXERCISE 3.7.2: Process state vs thread states.

(a) When a process implements ULTs, each TCB maintains a separate thread_state field but the process must also maintain a process_state field in the PCB since the kernel is not aware of the TCBs.
Which of the following combinations of process state and thread states could occur?

	PCB	TCB1	TCB2
1	blocked	ready	ready
2	ready	running	ready
3	blocked	blocked	blocked
4	running	running	running
5	ready	ready	blocked
6	running	running	blocked
7	running	running	ready

EXERCISE 3.7.3: Replicated scheduling information.

(a) In a process with ULTs, what would be the advantage of replicating the scheduling_information field from the PCB into every TCB?

3.8 Process interactions

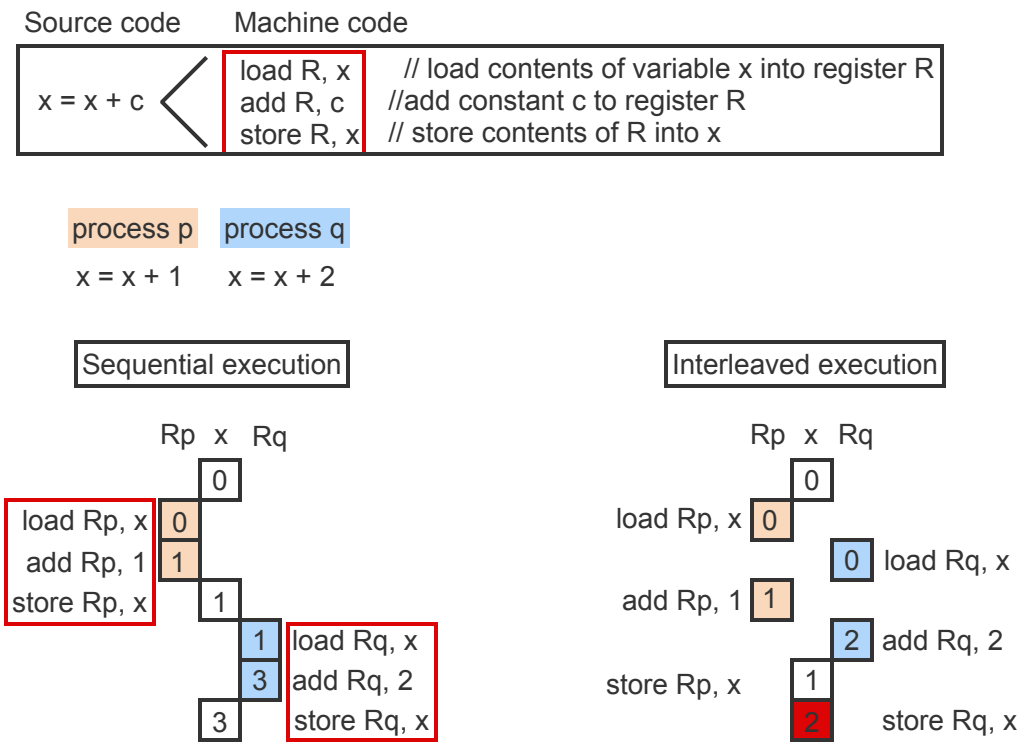
Process competition

Concurrency is the act of multiple processes (or threads) executing at the same time. When multiple physical CPUs are available, the processes may execute in parallel. On a single CPU, concurrency may be achieved by time-sharing.

When concurrent processes access a common data area, the data must be protected from simultaneous change by two or more processes. Otherwise the updated area may be left in an inconsistent state.

A ***critical section*** is a segment of code that cannot be entered by a process while another process is executing a corresponding segment of the code.

PARTICIPATION ACTIVITY 3.8.1: The critical section problem.



Animation content:

Static figure: A diagram of executing a critical section in two concurrent processes p and q. The top of the diagram shows a source instruction x = x + 1 and its translation into machine code as
load R, x // load contents of variable x into register R
add R, c //add constant c to register R
store R, x // store contents of R into x.
The rest of the diagram is divided into two columns: sequential execution and interleaved execution. In both cases, process p performs x = x + 1 while process q performs x = x + 2. Both also shows the contents of registers Rp and Rq and the shared variable x as they change through execution. Sequential execution shows the three instructions executed first by process p, then by process q with no interleaving. With interleaved execution, the instructions alternate between the two processes, causing an incorrect result.

Step 1: A source code instruction x = x + c is compiled into 3 machine-level instructions using a CPU register R. The top portion of the diagram appears.

Step 2: Process p executes the instruction x = x + 1. Process q executes the instruction x = x + 2.

Step 3: If the initial value of x is 0 and p executes the code first then p loads 0 into Rp, increments Rp by 1, and stores the result back into x. Sequential execution starts. Process p executes the 3 instructions. x = 1.

Step 4: If q executes next then q loads the updated value of x into Rq, increments Rq by 2, and stores the value back into x. The final value of x is 3 as expected.
Process q executes the 3 instructions. x = 3.

Step 5: Context switching can interleave the execution of p and q. First p loads the initial value of x = 0 into Rp. Interleaved execution starts. Process p executes the first instruction. Rp = 0.

Step 6: Next, q loads the same initial value x = 0 into Rq. Process q executes the first instruction. Rq = 0.

Step 7: Next, p increments Rp by 1 and q increments Rq by 2. Process p executes the add instruction. Rp = 1. Process q executes the add instruction. Rq = 2.

Step 8: Finally, p stores the value of Rp = 1 into x but q overwrites the value with Rq, which is 2. The update of x by p has been lost, resulting in an incorrect result.
Both processes execute the store instruction. X has the incorrect value 2.

Step 9: The sequence of the 3 instructions corresponding to the source code instruction x = x + c is a critical section and must not be interleaved.

Animation captions:

1. A source code instruction x = x + c is compiled into 3 machine-level instructions using a CPU register R.
2. Process p executes the instruction x = x + 1. Process q executes the instruction x = x + 2.
3. If the initial value of x is 0 and p executes the code first then p loads 0 into Rp, increments Rp by 1, and stores the result back into x.
4. If q executes next then q loads the updated value of x into Rq, increments Rq by 2, and stores the value back into x. The final value of x is 3 as expected.
5. Context switching can interleave the execution of p and q. First p loads the initial value of x = 0 into Rp.
6. Next, q loads the same initial value x = 0 into Rq.
7. Next, p increments Rp by 1 and q increments Rq by 2.
8. Finally, p stores the value of Rp = 1 into x but q overwrites the value with Rq, which is 2. The update of x by p has been lost, resulting in an incorrect result.
9. The sequence of the 3 instructions corresponding to the source code instruction x = x + c is a critical section and must not be interleaved.

PARTICIPATION ACTIVITY 3.8.2: Incorrect interleaving of code.

Processes p and q execute the instructions x = x + 1 and x = x + 2, respectively. Initially x = 0. After execution of the interleaved code, x = _____.

1)

```
p:              q:
load Rp,x
                load Rq,x
add Rp,1
                add Rq,2
                store Rq,x
store Rp,x
```

Check Show answer

2)

```
p:              q:
                load Rq,x
load Rp,x
add Rp,1
                add Rq,2
store Rp,x
                store Rq,x
```

Check Show answer

3)

```
p:              q:
load Rp,x
add Rp,1
                load Rq,x
                add Rq,2
                store Rq,x
store Rp,x
```

Check Show answer

A software solution to the CS problem.

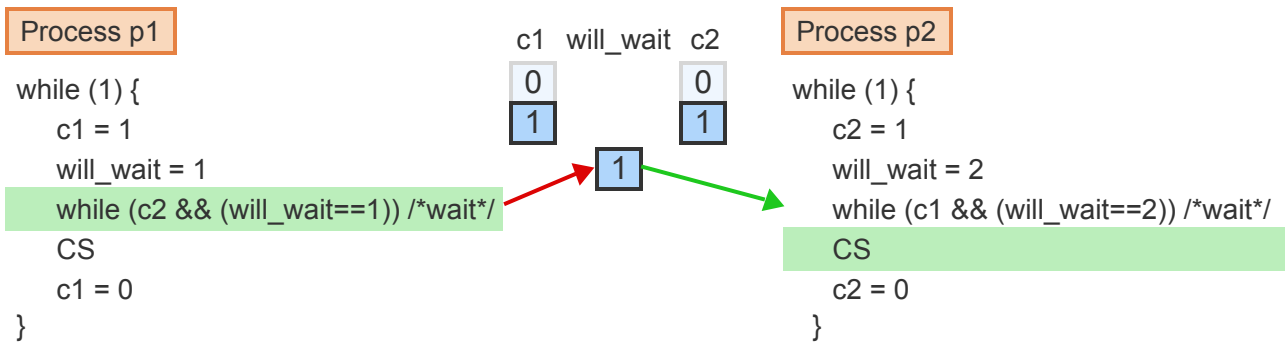
Any solution to the critical section (CS) problem must satisfy the following requirements:

1. Guarantee ***mutual exclusion***: Only one process may be executing within the CS.
2. Prevent ***lockout***: A process not attempting to enter the CS must not prevent other processes from entering the CS.

3. Prevent ***starvation***: A process (or a group of processes) must not be able to repeatedly enter the CS while other processes are waiting to enter.
4. Prevent ***deadlock***: Multiple processes trying to enter the CS at the same time must not block each other indefinitely.

In the following solution, a process indicates the interest to enter the CS by setting the variable c1 or c2 to 1. A third variable, will_wait, is used to break the tie if both processes try to enter CS simultaneously.

PARTICIPATION ACTIVITY 3.8.3: A software solution to the critical section problem.



Animation content:

Static figure: Two columns of pseudocode representing two processes, p1 and p2, which share variables c1, will_wait, and c2. The pseudocode outlines the while loop structure for both processes to ensure mutual exclusion in the critical section (CS). Both c1 and c2 are initialized to 0, and the initial value of will_wait is stated as irrelevant. Highlighted in each process's pseudocode is the critical section and the conditions that a process must wait for before entering the CS. The shared variables c1, will_wait, and c2 are visually represented in between the two pseudocode columns with their initial values set to 0.

Step 1: Processes p1 and p2 share a critical section CS. The initial values of c1 and c2 are 0. The initial value of will_wait is irrelevant. The code for both processes appears:
Process p1:

```
while (1) {
c1 = 1
will_wait = 1
while (c2 && (will_wait==1)) /*wait*/
CS
c1 = 0}
```

```
Process p2:
while (1) {
c2 = 1
will_wait = 2
while (c1 && (will_wait==2)) /*wait*/
CS
c2 = 0}
```

Step 2: Mutual exclusion is guaranteed. When both processes execute concurrently, c1 and c2 are both set to 1. But will_wait can only have one value, 1 or 2. The memory hardware does the writes in sequence. The first 2 instructions of both programs are highlighted. The value of will_wait is shown as "?".

Step 3: If p1 writes first, p2's subsequent write makes will_wait = 2. p1 wins the race and enters CS while p2 waits. CS in process 1 is highlighted. The wait loop in process 2 is highlighted.

Step 4: Deadlock is prevented for the same reason that guarantees mutual exclusion. When both processes enter the wait loop, one will always pass through because will_wait can only be 1 or 2.
The wait loop in both processes is highlighted. The value of will_wait is shown as "?".

Step 5: Lockout is also not possible. When p2 is not trying to enter CS, c2 = 0. Consequently, p1 may enter CS repeatedly, regardless of the value of will_wait.
Wait loop in process 1 is highlighted. An arrow points from c2=0 to the wait loop.

Step 6: Finally, starvation is also not possible. When p1 is in CS and p2 is in the wait loop, both c1 = c2 = 1 and will_wait = 2.

Step 7: If p1 wants to execute CS again, p1 must first set will_wait = 1.
Process 1 executes the code from the beginning, which sets will_wait = 1. Both processes are in the wait loops.

Step 8: Since will_wait = 1, p1 must wait while p2 enters CS. Thus p1 cannot starve p2 by entering CS repeatedly. CS in process 2 is highlighted, while process 1 remains in wait loop.

Animation captions:

1. Processes p1 and p2 share a critical section CS. The initial values of c1 and c2 are 0. The initial value of will_wait is irrelevant.
2. Mutual exclusion is guaranteed. When both processes execute concurrently, c1 and c2 are both set to 1. But will_wait can only have one value, 1 or 2. The memory hardware does the writes in sequence.
3. If p1 writes first, p2's subsequent write makes will_wait = 2. p1 wins the race and enters CS while p2 waits.

4. Deadlock is prevented for the same reason that guarantees mutual exclusion. When both processes enter the wait loop, one will always pass through because will_wait can only be 1 or 2.
5. Lockout is also not possible. When p2 is not trying to enter CS, c2 = 0. Consequently, p1 may enter CS repeatedly, regardless of the value of will_wait.
6. Finally, starvation is also not possible. When p1 is in CS and p2 is in the wait loop, both c1 = c2 = 1 and will_wait = 2.
7. If p1 wants to execute CS again, p1 must first set will_wait = 1.
8. Since will_wait = 1, p1 must wait while p2 enters CS. Thus p1 cannot starve p2 by entering CS repeatedly.

PARTICIPATION ACTIVITY 3.8.4: Possible values of the variables.

1) When p1 is in CS, the value of c1 can be 0.
   - True
   - False
2) When p1 is in CS, the value of will_wait can be 0.
   - True
   - False
3) When p1 is in CS, the value of will_wait can be 1.
   - True
   - False
4) When p1 is in CS, the value of will_wait can be 2.
   - True
   - False

PARTICIPATION ACTIVITY 3.8.5: An incorrect solution 1.

p1 executes the code:

```
while (1) {
   while (turn==2) /*wait*/
   CS
   turn = 2
```

p2 executes the code:

```
while (1) {
   while (turn==1) /*wait*/
   CS
   turn = 1
```

```
    ...
}
```

```
    ...
}
```

Assuming the variable turn is initialized to 1 or 2, which requirement is violated in the above solution?

- Mutual exclusion not guaranteed.
- Lockout is possible.
- Starvation is possible.
- Deadlock is possible.

PARTICIPATION ACTIVITY 3.8.6: An incorrect solution 2.

p1 executes the code:

```
while (1) {
   c1 = 1
   while (c2) /*wait*/
   CS
   c1 = 0
   ...
}
```

p2 executes the code:

```
while (1) {
   c2 = 1
   while (c1) /*wait*/
   CS
   c2 = 0
   ...
}
```

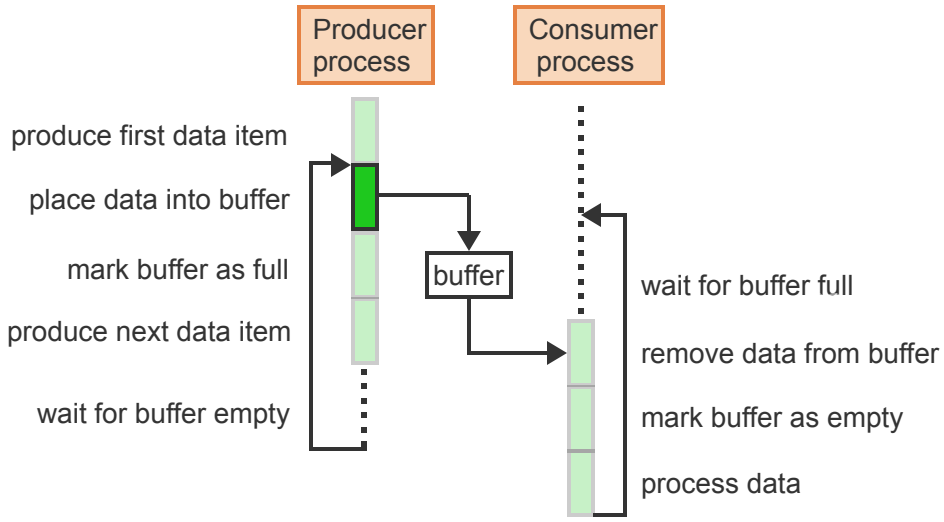
Which requirement is violated in the above solution?

- Mutual exclusion not guaranteed.
- Lockout is possible.
- Starvation is possible.
- Deadlock is possible.

Process cooperation

In addition to the CS problem, where processes compete for access to a shared resource, many applications require the cooperation of processes to solve a common goal. A typical scenario is when one process produces data needed by another process. The producer process must be able to inform the waiting process whenever a new data item is available. In turn, the producer must wait for an acknowledgement from the consumer, indicating that the consumer is ready to accept the next data item.

PARTICIPATION ACTIVITY 3.8.7: Producer-consumer synchronization.



Animation content:

Static figure: A Producer process and a Consumer process. Solid vertical lines represent execution while dotted lines represent waiting. Between the producer and the consumer is a buffer indicating the shared resource with arrows pointing from producer to buffer and from buffer to consumer.
The producer executes the operations: produce first data item, place data into buffer, mark buffer as full, produce next data item, and wait for buffer empty. Concurrently, the consumer executes the operations: wait for buffer full, remove data from buffer, mark buffer as empty, and process data.

Step 1: A producer and a consumer process cooperate by sharing a buffer, which can hold one data item.

Step 2: The producer produces the first data item, places the data item into the buffer, and marks the buffer as full.

Step 3: The consumer process must wait until the buffer has been filled. A dotted line appears in the consumer process while the producer executes.

Step 4: The producer can now produce the next data item but must not overwrite the buffer until the consumer has removed the previous data item. A dotted line in producer indicates waiting, before an arrow can take it back to placing the next data into buffer.

Step 5: The consumer removes the data item from the buffer and marks the buffer as empty. Solid line in consumer indicates the execution of the 2 operations.

Step 6: While the consumer processes the last data item, the producer can start placing the next data item into the buffer. The operation “place data into buffer” in the producer is highlighted concurrently with the operation “process data” in the consumer.

Step 7: After processing the data item, the consumer goes back to wait for the buffer to become full again to start the next production-consumption cycle. An arrow in consumer points back to the dotted line that represents waiting.
This concludes the cycle with the consumer being ready to receive the next data item as it has finished processing the previous one, and the buffer is ready to be filled again by the producer.

Animation captions:

1. A producer and a consumer process cooperate by sharing a buffer, which can hold one data item.
2. The producer produces the first data item, places the data item into the buffer, and marks the buffer as full.
3. The consumer process must wait until the buffer has been filled.
4. The producer can now produce the next data item but must not overwrite the buffer until the consumer has removed the previous data item.
5. The consumer removes the data item from the buffer and marks the buffer as empty.
6. While the consumer processes the last data item, the producer can start placing the next data item into the buffer.
7. After processing the data item, the consumer goes back to wait for the buffer to become full again to start the next production-consumption cycle.

PARTICIPATION ACTIVITY 3.8.8: Possible overlap between the producer and the consumer.

Which code segments could never be executing simultaneously?

- "produce next data item" and "remove data from buffer"
- "place data into buffer" and "process data"
- "mark buffer as full" and "remove data from buffer"

PARTICIPATION ACTIVITY 3.8.9: The consumer-producer problem vs the critical section problem.

1) The buffer is a critical section.

- True
- False

2) The code segments "place data into buffer" and "remove data from buffer" are critical sections because both access the shared buffer. Thus the software solution to the critical section problem is sufficient to also solve the producer-consumer problem.

- True
- False

EXERCISE 3.8.1: Possible code interleaving.

A process executes the sequence of machine-level instructions s1, s2, and s3. Another process executes the sequence r1 and r2.

(a) Determine all possible interleavings of the instructions when the processes run concurrently.

EXERCISE 3.8.2: Critical section.

The following program is executed by 2 processes p and q concurrently:

```
for i = 1; i <= 3; i++
   x = x + 1
```

- The variable x is shared by the two processes.
- The instruction x = x + 1 is compiled into 3 machine instructions to load x into a register, increment the register value, and store the register back into x.

(a) If x is initially 0, determine the expected value of x when both p and q terminate.

(b) In the absence of any synchronization primitives, program interleaving can result in a loss of 1 or more of the updates. Show one sequence instruction execution where the final value of x is less than the expected value.

(c) Determine the smallest possible value that x can have at the end of the program due to lost updates resulting from arbitrary code interleaving.

EXERCISE 3.8.3: Effect of interchanging the role of will_wait.

Modify the solution to the CS problem by reversing the will_wait assignment: p1 sets will_wait = 2 and p2 sets will_wait = 1. Thus each process, rather than expressing willingness to wait, states that the other process should wait if necessary.

(a) Which of the 4 requirements will be violated?

EXERCISE 3.8.4: Potential concurrency in the producer-consumer problem.

.

(a) Assume that the code segments "produce data item", "place data into buffer", "remove data from buffer", and "process data" all take the same time. Show which code segments will be executing concurrently for the first 3 cycles of filling/emptying the buffer.